1. Definition

This project is a full-stack OTP Verification System that verifies users through one-time passwords sent to email or phone. It combines a modern frontend dashboard with a Node.js and Express backend that supports OTP generation, OTP verification, expiry handling, and JWT-based access for protected APIs. The system also includes CRUD operations for users, where only OTP-verified contacts are allowed.

The project is designed with modular backend architecture, clean middleware layering, and practical real-world notification support using Nodemailer and Twilio. Persistence is handled through MongoDB with Mongoose models for both users and OTPs, so data created from the web app is stored in the database rather than temporary memory.

2. Programming languages/scripting languages/tools used

- JavaScript (Node.js): Used for backend API, business logic, and frontend scripting.
- HTML5: Used to build the structure of the frontend pages and forms.
- CSS3: Used for styling, responsive UI design, and animation effects.
- Node.js: Runtime for executing backend JavaScript code.
- Express.js: Framework used to build REST APIs.
- MongoDB: Database used to persist users and OTP records.
- Mongoose: ODM used for defining schemas and interacting with MongoDB.
- JSON Web Token (jsonwebtoken): Used for authentication and protected routes.
- express-validator: Used for validating incoming API request payloads.
- Nodemailer: Used to send OTP over email.
- Twilio: Used to send OTP over SMS.
- Morgan: HTTP request logger middleware.
- CORS: Enables secure cross-origin API requests.
- dotenv: Loads environment variables from .env file.
- nodemon: Restarts backend automatically during development.

3. paste all your codes with mentioning their file name on top.

3. Full Backend, Frontend & DB codes with File Name and Purpose

File Name: backend/package.json
Purpose: Backend package manifest containing scripts and dependencies.
~~~json
{
  "name": "otp-verification-backend",
  "version": "1.0.0",
  "description": "OTP verification system backend using Node.js and Express",
  "main": "src/server.js",
  "scripts": {
    "start": "node src/server.js",
    "dev": "nodemon src/server.js"
  },
  "keywords": [
    "otp",
~~~

```
    "express",
    "nodejs",
    "mongodb"
  ],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "express-validator": "^7.2.0",
    "jsonwebtoken": "^9.0.2",
    "mongoose": "^8.8.1",
    "morgan": "^1.10.0",
    "nodemailer": "^6.10.0",
    "twilio": "^5.4.2"
  },
  "devDependencies": {
    "nodemon": "^3.1.7"
  }
}
~~~
```

File Name: backend/.env.example
Purpose: Environment variable template required to run the backend.

```env
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/otp_verification_system
JWT_SECRET=change_this_secret
OTP_EXPIRY_MINUTES=5

# Nodemailer SMTP config (for email OTP)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your_email@gmail.com
SMTP_PASS=your_email_app_password
SMTP_FROM=your_email@gmail.com

# Twilio config (required for real SMS OTP delivery)
# Use E.164 format for TWILIO_PHONE_NUMBER, for example: +14155552671
TWILIO_ACCOUNT_SID=your_twilio_account_sid
TWILIO_AUTH_TOKEN=your_twilio_auth_token
TWILIO_PHONE_NUMBER=+12345678901
SMS_DEFAULT_COUNTRY_CODE=+91
```

File Name: backend/src/config/env.js
Purpose: Reads and exports application environment configuration.

~~~js
const dotenv = require('dotenv');

dotenv.config();

module.exports = {
  port: process.env.PORT || 5000,
  mongoUri: process.env.MONGO_URI || '',
  jwtSecret: process.env.JWT_SECRET || 'dev-secret-key',
  otpExpiryMinutes: Number(process.env.OTP_EXPIRY_MINUTES || 5),
  smtpHost: process.env.SMTP_HOST || '',
  smtpPort: Number(process.env.SMTP_PORT || 587),
  smtpUser: process.env.SMTP_USER || '',
  smtpPass: process.env.SMTP_PASS || '',
  smtpFrom: process.env.SMTP_FROM || process.env.SMTP_USER || '',
  twilioAccountSid: process.env.TWILIO_ACCOUNT_SID || '',
  twilioAuthToken: process.env.TWILIO_AUTH_TOKEN || '',
  twilioPhoneNumber: process.env.TWILIO_PHONE_NUMBER || '',
  smsDefaultCountryCode: process.env.SMS_DEFAULT_COUNTRY_CODE || '+91',
};
~~~

File Name: backend/src/config/db.js //Database Connection.
Purpose: Connects the backend to MongoDB through Mongoose.
~~~js
const mongoose = require('mongoose');
const env = require('./env');

const connectDB = async () => {
  if (!env.mongoUri) {
    throw new Error('MONGO_URI is not set');
  }

  await mongoose.connect(env.mongoUri);
  console.log('MongoDB connected successfully.');
};

module.exports = { connectDB };
~~~

File Name: backend/src/app.js
Purpose: Creates Express app, adds middleware, and mounts routes.
~~~js
const express = require('express');
const cors = require('cors');
const morgan = require('morgan');

const authRoutes = require('./routes/authRoutes');
const userRoutes = require('./routes/userRoutes');
~~~

```
const { notFoundHandler, errorHandler } = require('./middlewares/errorMiddleware');

const app = express();

app.use(cors());
app.use(express.json());
app.use(morgan('dev'));

app.get('/api/health', (req, res) => {
  res.status(200).json({ success: true, message: 'Server is running' });
});

app.use('/api/auth', authRoutes);
app.use('/api/users', userRoutes);

app.use(notFoundHandler);
app.use(errorHandler);

module.exports = app;
~~~

### File Name: backend/src/server.js
Purpose: Starts the backend server after DB connection.
~~~js
const app = require('./app');
const env = require('./config/env');
const { connectDB } = require('./config/db');

const startServer = async () => {
  try {
    await connectDB();

    app.listen(env.port, () => {
      console.log(`Server listening on port ${env.port}`);
    });
  } catch (error) {
    console.error('Failed to start server:', error.message);
    process.exit(1);
  }
};

startServer();
~~~

### File Name: backend/src/models/User.js
Purpose: Defines User schema and model.
~~~js
const mongoose = require('mongoose');
```

```
const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true,
    },
    contact: {
      type: String,
      required: true,
      unique: true,
      trim: true,
    },
    contactType: {
      type: String,
      enum: ['email', 'phone'],
      required: true,
    },
    isVerified: {
      type: Boolean,
      default: false,
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model('User', userSchema);
~~~


### File Name: backend/src/models/Otp.js
Purpose: Defines OTP schema and model.
~~~js
const mongoose = require('mongoose');

const otpSchema = new mongoose.Schema(
  {
    contact: {
      type: String,
      required: true,
      trim: true,
      index: true,
    },
    otp: {
      type: String,
      required: true,
    },
    expiresAt: {
      type: Date,
      required: true,
```

```
    },
    isUsed: {
      type: Boolean,
      default: false,
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model('Otp', otpSchema);
~~~

### File Name: backend/src/utils/ApiError.js
Purpose: Standardized API error class.
~~~js
class ApiError extends Error {
  constructor(statusCode, message) {
    super(message);
    this.statusCode = statusCode;
  }
}

module.exports = ApiError;
~~~

### File Name: backend/src/utils/generateOtp.js
Purpose: Generates 6-digit numeric OTP.
~~~js
const generateOtp = () => {
  return Math.floor(100000 + Math.random() * 900000).toString();
};

module.exports = generateOtp;
~~~

### File Name: backend/src/middlewares/authMiddleware.js
Purpose: Validates JWT token for protected routes.
~~~js
const jwt = require('jsonwebtoken');
const env = require('../config/env');
const ApiError = require('../utils/ApiError');

const protect = (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return next(new ApiError(401, 'Authentication token missing'));
  }
```

```
  const token = authHeader.split(' ')[1];

  try {
    const decoded = jwt.verify(token, env.jwtSecret);
    req.user = decoded;
    next();
  } catch (error) {
    next(new ApiError(401, 'Invalid or expired token'));
  }
};

module.exports = { protect };
~~~


### File Name: backend/src/middlewares/validateMiddleware.js
Purpose: Formats and returns validation errors from express-validator.
~~~js
const { validationResult } = require('express-validator');

const validateRequest = (req, res, next) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({
      success: false,
      message: 'Validation failed',
      errors: errors.array(),
    });
  }
  next();
};

module.exports = { validateRequest };
~~~


### File Name: backend/src/middlewares/errorMiddleware.js
Purpose: Handles unknown routes and global API errors.
~~~js
const ApiError = require('../utils/ApiError');

const notFoundHandler = (req, res, next) => {
  next(new ApiError(404, `Route not found: ${req.originalUrl}`));
};

const errorHandler = (err, req, res, next) => {
  const statusCode = err.statusCode || 500;
  const message = err.message || 'Internal server error';

  res.status(statusCode).json({
    success: false,
```

```
    message,
  });
};

module.exports = { notFoundHandler, errorHandler };
~~~

### File Name: backend/src/validators/authValidators.js
Purpose: Validates generate and verify OTP request payloads.
~~~js
const { body } = require('express-validator');

const contactValidator = body('contact')
  .notEmpty()
  .withMessage('Contact is required')
  .isString()
  .withMessage('Contact must be a string')
  .customSanitizer((value) => String(value || '').trim())
  .custom((value) => {
    const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    const phonePattern = /^\+?[0-9\s()-]{10,18}$/;

    if (emailPattern.test(value) || phonePattern.test(value)) {
      return true;
    }

    throw new Error('Contact must be a valid email or phone number');
  });

const otpValidator = body('otp')
  .notEmpty()
  .withMessage('OTP is required')
  .matches(/^\d{6}$/)
  .withMessage('OTP must be a 6-digit number');

const generateOtpValidation = [contactValidator];
const verifyOtpValidation = [contactValidator, otpValidator];

module.exports = { generateOtpValidation, verifyOtpValidation };
~~~

### File Name: backend/src/validators/userValidators.js
Purpose: Validates user CRUD request fields and params.
~~~js
const { body, param } = require('express-validator');

const contactTypeValidator = body('contactType')
  .notEmpty()
  .withMessage('contactType is required')
```

```
  .isIn(['email', 'phone'])
  .withMessage('contactType must be email or phone');

const createUserValidation = [
  body('name').notEmpty().withMessage('Name is required').isString().trim(),
  body('contact').notEmpty().withMessage('Contact is required').isString().trim(),
  contactTypeValidator,
];

const updateUserValidation = [
  param('id').notEmpty().withMessage('User ID is required'),
  body('name').optional().isString().withMessage('Name must be a string').trim(),
  body('contact').optional().isString().withMessage('Contact must be a string').trim(),
  body('contactType').optional().isIn(['email', 'phone']).withMessage('contactType
must be email or phone'),
];

const userIdValidation = [param('id').notEmpty().withMessage('User ID is required')];

module.exports = { createUserValidation, updateUserValidation, userIdValidation };
~~~


### File Name: backend/src/services/otpService.js
Purpose: Handles OTP creation, lookup, usage marking, and expiry cleanup.
~~~js
const { useMongo } = require('../config/db');
const Otp = require('../models/Otp');
const { memoryOtps } = require('../data/memoryStore');

const createOtpEntry = async ({ contact, otp, expiresAt }) => {
  if (useMongo()) {
    await Otp.deleteMany({ contact, isUsed: false });
    return Otp.create({ contact, otp, expiresAt, isUsed: false });
  }

  for (let i = memoryOtps.length - 1; i >= 0; i -= 1) {
    if (memoryOtps[i].contact === contact && !memoryOtps[i].isUsed) {
      memoryOtps.splice(i, 1);
    }
  }

  const otpEntry = {
    id: Date.now().toString(),
    contact,
    otp,
    expiresAt,
    isUsed: false,
  };
  memoryOtps.push(otpEntry);
```

```
  return otpEntry;
};

const getLatestUnusedOtpByContact = async (contact) => {
  if (useMongo()) {
    return Otp.findOne({ contact, isUsed: false }).sort({ createdAt: -1 });
  }

  const filtered = memoryOtps
    .filter((item) => item.contact === contact && !item.isUsed)
    .sort((a, b) => new Date(b.expiresAt) - new Date(a.expiresAt));
  return filtered[0] || null;
};

const markOtpAsUsed = async (otpEntry) => {
  if (!otpEntry) return;

  if (useMongo()) {
    otpEntry.isUsed = true;
    await otpEntry.save();
    return;
  }

  otpEntry.isUsed = true;
};

const cleanupExpiredOtps = async () => {
  const now = new Date();

  if (useMongo()) {
    await Otp.deleteMany({ expiresAt: { $lt: now } });
    return;
  }

  for (let i = memoryOtps.length - 1; i >= 0; i -= 1) {
    if (new Date(memoryOtps[i].expiresAt) < now) {
      memoryOtps.splice(i, 1);
    }
  }
};

module.exports = {
  createOtpEntry,
  getLatestUnusedOtpByContact,
  markOtpAsUsed,
  cleanupExpiredOtps,
};
~~~
```

File Name: backend/src/services/notificationService.js
Purpose: Sends OTP notifications through email (Nodemailer) or SMS (Twilio).
~~~js
const nodemailer = require('nodemailer');
const twilio = require('twilio');

const env = require('../config/env');

let emailTransporter;
let smsClient;

const getEmailTransporter = () => {
  if (emailTransporter) return emailTransporter;

  if (!env.smtpHost || !env.smtpPort || !env.smtpUser || !env.smtpPass
|| !env.smtpFrom) {
    return null;
  }

  emailTransporter = nodemailer.createTransport({
    host: env.smtpHost,
    port: env.smtpPort,
    secure: env.smtpPort === 465,
    auth: {
      user: env.smtpUser,
      pass: env.smtpPass,
    },
  });

  return emailTransporter;
};

const getSmsClient = () => {
  if (smsClient) return smsClient;

  if (!env.twilioAccountSid || !env.twilioAuthToken || !env.twilioPhoneNumber) {
    return null;
  }

  smsClient = twilio(env.twilioAccountSid, env.twilioAuthToken);
  return smsClient;
};

const normalizePhoneNumber = (contact) => {
  if (contact.startsWith('+')) return contact;
  return `${env.smsDefaultCountryCode}${contact}`;
};

const buildOtpMessage = (otp, expiresAt) => {
~~~

```
  const expiresAtText = new Date(expiresAt).toLocaleString();
  return [
    'OTP Verification Code',
    '',
    `Your one-time password is: ${otp}`,
    `This code expires at: ${expiresAtText}`,
    '',
    'If you did not request this code, please ignore this message.',
    'Do not share this OTP with anyone.',
  ].join('\n');
};

const buildOtpEmailHtml = ({ otp, expiresAt, contact }) => {
  const expiresAtText = new Date(expiresAt).toLocaleString();

  return `
  <!DOCTYPE html>
  <html lang="en">
    <head>
      <meta charset="UTF-8" />
      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
      <title>OTP Verification Code</title>
    </head>
    <body style="margin:0;padding:0;background:#f4f6fb;font-family:Segoe
UI,Arial,sans-serif;color:#1f2937;">
      <table role="presentation" width="100%" cellspacing="0" cellpadding="0"
border="0" style="background:#f4f6fb;padding:24px 12px;">
        <tr>
          <td align="center">
            <table role="presentation" width="100%" cellspacing="0" cellpadding="0"
border="0" style="max-width:620px;background:#ffffff;border-
radius:16px;overflow:hidden;border:1px solid #e5e7eb;box-shadow:0 12px 36px
rgba(15,23,42,0.12);">
              <tr>
                <td style="padding:28px 32px;background:linear-
gradient(135deg,#0f8b7d,#14b8a6);color:#ffffff;">
                  <p style="margin:0;font-size:12px;letter-spacing:1.4px;text-
transform:uppercase;opacity:0.9;">Secure Verification</p>
                  <h1 style="margin:8px 0 0;font-size:24px;line-height:1.2;font-
weight:700;">Your OTP Code</h1>
                </td>
              </tr>

              <tr>
                <td style="padding:30px 32px 10px;">
                  <p style="margin:0 0 14px;font-size:15px;line-height:1.65;">Hi,</p>
                  <p style="margin:0 0 22px;font-size:15px;line-height:1.65;">
                    Use the verification code below to complete your login or account
action for
```

```
                  <strong>${contact}</strong>.
                </p>

                <table role="presentation" width="100%" cellspacing="0"
cellpadding="0" border="0" style="margin:0 0 22px;">
                  <tr>
                    <td align="center" style="padding:20px;border:1px solid
#ccfbf1;background:#f0fdfa;border-radius:14px;">
                      <p style="margin:0 0 8px;font-size:12px;color:#0f766e;text-
transform:uppercase;letter-spacing:1.2px;font-weight:700;">One-Time Password</p>
                      <p style="margin:0;font-size:36px;line-height:1;letter-
spacing:10px;font-weight:800;color:#0f172a;">${otp}</p>
                    </td>
                  </tr>
                </table>

                <table role="presentation" width="100%" cellspacing="0"
cellpadding="0" border="0" style="margin:0 0 20px;">
                  <tr>
                    <td style="padding:14px 16px;background:#fff7ed;border:1px solid
#fed7aa;border-radius:10px;font-size:13px;line-height:1.5;color:#9a3412;">
                      This OTP is valid until <strong>${expiresAtText}</strong>.
                    </td>
                  </tr>
                </table>

                <p style="margin:0 0 10px;font-size:13px;line-
height:1.6;color:#6b7280;">
                  Security tip: Never share this OTP with anyone, including support
staff.
                </p>
              </td>
            </tr>

            <tr>
              <td style="padding:18px 32px 26px;border-top:1px solid #f1f5f9;">
                <p style="margin:0;font-size:12px;line-height:1.6;color:#94a3b8;">
                  If you did not request this code, you can safely ignore this email.
                </p>
              </td>
            </tr>
          </table>
        </td>
      </tr>
    </table>
  </body>
</html>
`;
};
```

```
const sendEmailOtp = async ({ contact, otp, expiresAt }) => {
  const transporter = getEmailTransporter();

  if (!transporter) {
    console.warn('SMTP is not fully configured. OTP will be logged instead of
emailed.');
    console.log(`OTP for ${contact}: ${otp} (expires at ${new
Date(expiresAt).toISOString()})`);
    return;
  }

  const message = buildOtpMessage(otp, expiresAt);
  const htmlMessage = buildOtpEmailHtml({ otp, expiresAt, contact });

  await transporter.sendMail({
    from: `OTP Verification System <${env.smtpFrom}>`,
    to: contact,
    subject: 'Your Premium OTP Verification Code',
    text: message,
    html: htmlMessage,
  });
};

const sendSmsOtp = async ({ contact, otp, expiresAt }) => {
  const client = getSmsClient();

  if (!client) {
    throw new Error(
      'SMS delivery is not configured. Please set TWILIO_ACCOUNT_SID,
TWILIO_AUTH_TOKEN, and TWILIO_PHONE_NUMBER in backend/.env',
    );
  }

  const to = normalizePhoneNumber(contact);
  const message = buildOtpMessage(otp, expiresAt);

  try {
    await client.messages.create({
      body: message,
      from: env.twilioPhoneNumber,
      to,
    });
  } catch (error) {
    throw new Error(`Failed to send SMS OTP: ${error.message}`);
  }
};

const sendOtpNotification = async ({ contact, contactType, otp, expiresAt }) => {
```

```
  if (contactType === 'email') {
    await sendEmailOtp({ contact, otp, expiresAt });
    return;
  }

  if (contactType === 'phone') {
    await sendSmsOtp({ contact, otp, expiresAt });
    return;
  }

  throw new Error('Unsupported contact type for OTP delivery');
};

module.exports = {
  sendOtpNotification,
};
~~~


### File Name: backend/src/controllers/authController.js
Purpose: Contains OTP generation and OTP verification controller actions.
~~~js
const jwt = require('jsonwebtoken');

const env = require('../config/env');
const User = require('../models/User');
const generateOtp = require('../utils/generateOtp');
const ApiError = require('../utils/ApiError');
const { sendOtpNotification } = require('../services/notificationService');
const {
  createOtpEntry,
  getLatestUnusedOtpByContact,
  markOtpAsUsed,
  cleanupExpiredOtps,
} = require('../services/otpService');

const normalizePhoneContact = (value) => {
  const compact = value.replace(/[\s()-]/g, '');

  if (compact.startsWith('+')) return compact;

  if (/^\d{10}$/.test(compact)) {
    return `${env.smsDefaultCountryCode}${compact}`;
  }

  return compact;
};

const normalizeContact = (contact) => {
  const trimmed = String(contact || '').trim();
```

```
  if (trimmed.includes('@')) {
    return trimmed.toLowerCase();
  }

  return normalizePhoneContact(trimmed);
};

const detectContactType = (contact) => {
  const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  const phonePattern = /^\+[0-9]{10,15}$|^[0-9]{10,15}$/;

  if (emailPattern.test(contact)) return 'email';
  if (phonePattern.test(contact)) return 'phone';
  return null;
};

const findUserByContact = async (contact) => {
  return User.findOne({ contact });
};

const generateOtpHandler = async (req, res, next) => {
  try {
    const contact = normalizeContact(req.body.contact);
    const contactType = detectContactType(contact);

    if (!contactType) {
      return next(new ApiError(400, 'Please provide a valid email or phone number'));
    }

    let user = await findUserByContact(contact);

    if (!user) {
      user = await User.create({
        name: 'OTP User',
        contact,
        contactType,
        isVerified: false,
      });
    }

    const otp = generateOtp();
    const expiresAt = new Date(Date.now() + env.otpExpiryMinutes * 60 * 1000);

    await createOtpEntry({ contact, otp, expiresAt });
    await sendOtpNotification({ contact, contactType, otp, expiresAt });

    res.status(200).json({
      success: true,
```

```
      message: 'OTP generated and sent successfully',
      data: {
        contact,
        expiresAt,
        // Exposing OTP for demo/testing. Remove this in production.
        otp,
      },
    });
  } catch (error) {
    next(error);
  }
};

const verifyOtpHandler = async (req, res, next) => {
  try {
    await cleanupExpiredOtps();

    const contact = normalizeContact(req.body.contact);
    const { otp } = req.body;
    const otpEntry = await getLatestUnusedOtpByContact(contact);

    if (!otpEntry) {
      return next(new ApiError(400, 'No active OTP found. Please generate OTP
again.'));
    }

    if (new Date(otpEntry.expiresAt) < new Date()) {
      return next(new ApiError(400, 'OTP expired. Please generate a new OTP.'));
    }

    if (otpEntry.otp !== otp) {
      return next(new ApiError(400, 'Invalid OTP'));
    }

    await markOtpAsUsed(otpEntry);

    let user = await findUserByContact(contact);

    if (user) {
      user.isVerified = true;
      await user.save();
    }

    const token = jwt.sign({ contact }, env.jwtSecret, { expiresIn: '1h' });

    res.status(200).json({
      success: true,
      message: 'OTP verified. Access granted.',
      token,
```

```
    });
  } catch (error) {
    next(error);
  }
};

module.exports = {
  generateOtpHandler,
  verifyOtpHandler,
};
~~~


### File Name: backend/src/controllers/userController.js
Purpose: Handles protected CRUD operations for verified users.
~~~js
const User = require('../models/User');
const ApiError = require('../utils/ApiError');

const listUsers = async (req, res, next) => {
  try {
    const users = await User.find({ isVerified: true }).sort({ createdAt: -1 });
    return res.status(200).json({ success: true, data: users });
  } catch (error) {
    next(error);
  }
};

const getUserById = async (req, res, next) => {
  try {
    const { id } = req.params;

    const user = await User.findById(id);
    if (!user) return next(new ApiError(404, 'User not found'));
    if (!user.isVerified) return next(new ApiError(403, 'Only verified users are
allowed in CRUD'));

    return res.status(200).json({ success: true, data: user });
  } catch (error) {
    next(error);
  }
};

const createUser = async (req, res, next) => {
  try {
    const { name, contact, contactType } = req.body;

    const existingUser = await User.findOne({ contact });
    if (!existingUser || !existingUser.isVerified) {
```

```
      return next(new ApiError(403, 'Contact must be OTP-verified before it can be
added to CRUD'));
    }

    if (existingUser.name !== 'OTP User') {
      return next(new ApiError(400, 'Contact already exists. Use update API to modify
user details'));
    }

    existingUser.name = name;
    existingUser.contactType = contactType;
    await existingUser.save();

    return res.status(201).json({
      success: true,
      message: 'Verified user added to CRUD successfully',
      data: existingUser,
    });
  } catch (error) {
    next(error);
  }
};

const updateUser = async (req, res, next) => {
  try {
    const { id } = req.params;

    const user = await User.findById(id);
    if (!user) return next(new ApiError(404, 'User not found'));
    if (!user.isVerified) return next(new ApiError(403, 'Only verified users are
allowed in CRUD'));

    Object.assign(user, req.body);
    const updatedUser = await user.save();

    return res.status(200).json({ success: true, message: 'User updated', data:
updatedUser });
  } catch (error) {
    next(error);
  }
};

const deleteUser = async (req, res, next) => {
  try {
    const { id } = req.params;

    const user = await User.findById(id);
    if (!user) return next(new ApiError(404, 'User not found'));
```

```
    if (!user.isVerified) return next(new ApiError(403, 'Only verified users are
allowed in CRUD'));

    await user.deleteOne();

    return res.status(200).json({ success: true, message: 'User deleted' });
  } catch (error) {
    next(error);
  }
};

module.exports = {
  listUsers,
  getUserById,
  createUser,
  updateUser,
  deleteUser,
};
~~~


### File Name: backend/src/routes/authRoutes.js
Purpose: Registers OTP auth routes.
~~~js
const express = require('express');

const { generateOtpHandler, verifyOtpHandler } =
require('../controllers/authController');
const { validateRequest } = require('../middlewares/validateMiddleware');
const { generateOtpValidation, verifyOtpValidation } =
require('../validators/authValidators');

const router = express.Router();

router.post('/generate-otp', generateOtpValidation, validateRequest,
generateOtpHandler);
router.post('/verify-otp', verifyOtpValidation, validateRequest, verifyOtpHandler);

module.exports = router;
~~~


### File Name: backend/src/routes/userRoutes.js
Purpose: Registers protected user CRUD routes.
~~~js
const express = require('express');

const {
  listUsers,
  getUserById,
  createUser,
```

```
  updateUser,
  deleteUser,
} = require('../controllers/userController');
const { protect } = require('../middlewares/authMiddleware');
const { validateRequest } = require('../middlewares/validateMiddleware');
const { createUserValidation, updateUserValidation, userIdValidation } =
require('../validators/userValidators');

const router = express.Router();

router.use(protect);

router.get('/', listUsers);
router.get('/:id', userIdValidation, validateRequest, getUserById);
router.post('/', createUserValidation, validateRequest, createUser);
router.put('/:id', updateUserValidation, validateRequest, updateUser);
router.delete('/:id', userIdValidation, validateRequest, deleteUser);

module.exports = router;
~~~


### File Name: frontend/index.html
Purpose: Builds the UI layout and form components for OTP flow and CRUD actions.
~~~html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>OTP Verification System</title>
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link

href="https://fonts.googleapis.com/css2?family=Manrope:wght@400;600;700;800&display=sw
ap"
      rel="stylesheet"
    />
    <link
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      rel="stylesheet"
      integrity="sha384-
QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALEwIH"
      crossorigin="anonymous"
    />
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>
    <div class="bg-orb bg-orb-1" aria-hidden="true"></div>
```

```
    <div class="bg-orb bg-orb-2" aria-hidden="true"></div>

    <main class="container py-4 py-md-5">
      <section class="hero card-soft p-4 p-lg-5 mb-4">
        <div class="d-flex flex-column flex-lg-row gap-3 align-items-lg-center
justify-content-between">
          <div>
            <p class="eyebrow mb-2">Secure Verification Flow</p>
            <h1 class="display-6 fw-bold mb-2">OTP Verification System</h1>
            <p class="subtitle mb-0">
              Generate OTP, verify users, and call protected CRUD APIs in one elegant
dashboard.
            </p>
          </div>
          <div class="token-pill" id="tokenStatus">Token: Not Verified</div>
        </div>
      </section>

      <div class="row g-4">
        <div class="col-12 col-lg-6">
          <section class="card-soft p-4 h-100">
            <h2 class="h4 mb-3">
              <span class="step-badge">1</span> Generate OTP
            </h2>
            <form id="generateForm" class="vstack gap-3">
              <div>
                <label class="form-label" for="contact">Email or Phone</label>
                <input
                  class="form-control form-control-lg"
                  id="contact"
                  name="contact"
                  type="text"
                  inputmode="text"
                  autocomplete="username"
                  placeholder="example@mail.com or +911234567890"
                  required
                />
              </div>
              <button class="btn btn-brand btn-lg w-100" type="submit">
                <span class="spinner-icon">âš¡</span> Generate OTP
              </button>
            </form>
          </section>
        </div>

        <div class="col-12 col-lg-6">
          <section class="card-soft p-4 h-100">
            <h2 class="h4 mb-3">
              <span class="step-badge">2</span> Verify OTP
```

```
            </h2>
            <form id="verifyForm" class="vstack gap-3">
              <div>
                <label class="form-label" for="verifyContact">Email or Phone</label>
                <input
                  class="form-control form-control-lg"
                  id="verifyContact"
                  name="contact"
                  type="text"
                  inputmode="text"
                  autocomplete="username"
                  required
                />
              </div>

              <div>
                <label class="form-label" for="otp">OTP Code</label>
                <input
                  class="form-control form-control-lg otp-input"
                  id="otp"
                  name="otp"
                  type="text"
                  maxlength="6"
                  placeholder="000000"
                  required
                />
              </div>

              <button class="btn btn-success btn-lg w-100" type="submit">
                <span class="spinner-icon">âœ“</span> Verify OTP
              </button>
            </form>
          </section>
        </div>

        <div class="col-12 col-xl-6">
          <section class="card-soft p-4 h-100">
            <div class="d-flex justify-content-between align-items-center mb-3">
              <h2 class="h4 mb-0">
                <span class="step-badge">3</span> User CRUD (Protected)
              </h2>
              <button id="fetchUsersBtn" class="btn btn-sm btn-outline-brand"
type="button">ðŸ”„ Fetch Users</button>
            </div>
            <p class="small text-secondary mb-3">
              âœ“ Verify OTP first, then create user with the same verified
email/phone.
            </p>
```

```
            <form id="createUserForm" class="row g-3">
              <div class="col-12 col-md-6">
                <label class="form-label" for="name">Full Name</label>
                <input class="form-control" id="name" name="name" type="text"
placeholder="John Doe" required />
              </div>
              <div class="col-12 col-md-6">
                <label class="form-label" for="userContact">Email or Phone</label>
                <input
                  class="form-control"
                  id="userContact"
                  name="contact"
                  type="text"
                  placeholder="verified contact"
                  required
                />
              </div>
              <div class="col-12">
                <label class="form-label" for="contactType">Contact Type</label>
                <select class="form-select" id="contactType" name="contactType"
required>
                  <option value="">-- Select --</option>
                  <option value="email">Email</option>
                  <option value="phone">Phone</option>
                </select>
              </div>
              <div class="col-12">
                <button class="btn btn-brand w-100" type="submit">âž• Create
User</button>
              </div>
            </form>
          </section>
        </div>

        <div class="col-12 col-xl-6">
          <section class="card-soft p-4 h-100">
            <div class="d-flex justify-content-between align-items-center mb-3">
              <h2 class="h4 mb-0">Live Response</h2>
              <button id="clearLogsBtn" class="btn btn-sm btn-outline-secondary"
type="button">ðŸ—‘ Clear</button>
            </div>
            <pre id="output" class="output-box mb-0">Ready.</pre>
          </section>
        </div>

        <div class="col-12">
          <section class="card-soft p-4">
            <div class="d-flex justify-content-between align-items-center mb-3">
              <h2 class="h4 mb-0">ðŸ“œ Verification History</h2>
```

```
            <span class="badge bg-brand" id="logCount">0</span>
          </div>
          <div id="logsContainer" class="logs-list">
            <p class="text-secondary text-center py-4">No verifications yet. Start
by generating an OTP.</p>
          </div>
        </section>
      </div>
    </div>
  </main>

  <script

src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
    integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
    crossorigin="anonymous"
  ></script>
  <script src="script.js"></script>
 </body>
</html>
~~~

### File Name: frontend/style.css
Purpose: Controls complete frontend visual design, responsiveness, and animations.
~~~css
:root {
  --bg: #f8f6f3;
  --text: #1a1410;
  --muted: #6b5f53;
  --brand: #0f8b7d;
  --brand-deep: #086e61;
  --brand-light: #20c997;
  --brand-accent: #14d8a6;
  --success: #10b981;
  --warning: #f59e0b;
  --danger: #ef4444;
  --card: rgba(255, 255, 255, 0.75);
  --card-border: rgba(100, 88, 75, 0.12);
  --output-bg: #0f0f0f;
  --output-text: #fef3c7;
  --shadow-sm: 0 2px 8px rgba(15, 10, 5, 0.08);
  --shadow-md: 0 8px 24px rgba(15, 10, 5, 0.12);
  --shadow-lg: 0 20px 48px rgba(15, 10, 5, 0.16);
}

* {
  box-sizing: border-box;
}
```

```
body {
  margin: 0;
  min-height: 100vh;
  font-family: "Manrope", sans-serif;
  color: var(--text);
  background:
    radial-gradient(circle at 15% 20%, rgba(15, 139, 125, 0.25), transparent 35%),
    radial-gradient(circle at 85% 15%, rgba(32, 201, 151, 0.15), transparent 45%),
    radial-gradient(circle at 50% 100%, rgba(15, 139, 125, 0.1), transparent 50%),
    linear-gradient(135deg, #f9f5f0 0%, #f0f9f7 50%, #eef5ff 100%);
  overflow-x: hidden;
}

.bg-orb {
  position: fixed;
  border-radius: 999px;
  filter: blur(30px);
  z-index: -1;
  opacity: 0.65;
  animation: drift 11s ease-in-out infinite alternate;
}

.bg-orb-1 {
  width: 320px;
  height: 320px;
  top: -80px;
  right: -70px;
  background: rgba(13, 148, 136, 0.35);
}

.bg-orb-2 {
  width: 300px;
  height: 300px;
  left: -70px;
  bottom: -80px;
  background: rgba(251, 191, 36, 0.34);
  animation-duration: 14s;
}

.card-soft {
  background: var(--card);
  border: 1px solid var(--card-border);
  border-radius: 1.25rem;
  box-shadow: var(--shadow-md);
  -webkit-backdrop-filter: blur(8px);
  backdrop-filter: blur(8px);
  animation: riseIn 0.7s cubic-bezier(0.34, 1.56, 0.64, 1) both;
  transition: all 0.3s cubic-bezier(0.34, 1.56, 0.64, 1);
```

```
}

.card-soft:hover {
  box-shadow: var(--shadow-lg);
  transform: translateY(-2px);
}

.hero {
  border-left: 8px solid var(--brand);
}

.eyebrow {
  color: var(--brand-deep);
  text-transform: uppercase;
  letter-spacing: 0.08em;
  font-weight: 700;
  font-size: 0.8rem;
}

.subtitle {
  max-width: 58ch;
  color: var(--muted);
}

.token-pill {
  background: #fff;
  color: var(--brand-deep);
  border: 1px solid rgba(15, 118, 110, 0.35);
  padding: 0.55rem 0.95rem;
  border-radius: 999px;
  font-weight: 700;
  font-size: 0.92rem;
  white-space: nowrap;
}

.token-pill.is-authenticated {
  background: rgba(13, 148, 136, 0.12);
}

.form-control,
.form-select {
  border-radius: 0.9rem;
  border: 2px solid rgba(120, 113, 108, 0.15);
  padding: 0.85rem 1rem;
  font-size: 1rem;
  font-weight: 500;
  transition: all 0.3s ease;
  background: rgba(255, 255, 255, 0.6);
}
```

```
.form-control:hover,
.form-select:hover {
  border-color: rgba(15, 139, 125, 0.3);
  background: rgba(255, 255, 255, 0.8);
}

.form-control:focus,
.form-select:focus {
  border-color: var(--brand);
  background: rgba(255, 255, 255, 0.95);
  box-shadow: 0 0 0 0.3rem rgba(15, 139, 125, 0.12), inset 0 0 0 2px rgba(15, 139, 125,
0.05);
  outline: none;
}

.btn-brand {
  background: linear-gradient(135deg, var(--brand) 0%, var(--brand-light) 50%, var(--
brand-accent) 100%);
  color: #fff;
  border: none;
  font-weight: 700;
  font-size: 1rem;
  letter-spacing: 0.02em;
  box-shadow: var(--shadow-md);
  transition: all 0.3s cubic-bezier(0.34, 1.56, 0.64, 1);
  position: relative;
  overflow: hidden;
}

.btn-brand::before {
  content: '';
  position: absolute;
  top: 50%;
  left: 50%;
  width: 0;
  height: 0;
  border-radius: 50%;
  background: rgba(255, 255, 255, 0.3);
  transform: translate(-50%, -50%);
  transition: width 0.6s, height 0.6s;
}

.btn-brand:hover::before {
  width: 300px;
  height: 300px;
}

.btn-brand:hover,
```

```
.btn-brand:focus {
  color: #fff;
  background: linear-gradient(135deg, var(--brand-deep) 0%, var(--brand) 50%, var(--
brand-light) 100%);
  box-shadow: 0 12px 32px rgba(15, 139, 125, 0.35);
  transform: translateY(-3px);
}

.btn-outline-brand {
  border-color: var(--brand);
  color: var(--brand);
}

.btn-outline-brand:hover {
  background: var(--brand);
  border-color: var(--brand);
  color: #fff;
}

.otp-input {
  letter-spacing: 0.2em;
  font-weight: 800;
}

.output-box {
  min-height: 350px;
  background: linear-gradient(170deg, var(--output-bg), #292524);
  color: var(--output-text);
  border-radius: 0.95rem;
  border: 1px solid rgba(245, 158, 11, 0.32);
  padding: 1rem;
  overflow-x: auto;
  white-space: pre-wrap;
  word-break: break-word;
}

@media (max-width: 768px) {
  .hero {
    border-left-width: 5px;
  }

  .output-box {
    min-height: 260px;
  }
}

@keyframes riseIn {
  from {
    opacity: 0;
```

```
    transform: translateY(20px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

@keyframes drift {
  from {
    transform: translateY(0) translateX(0) scale(1);
  }
  to {
    transform: translateY(-25px) translateX(18px) scale(1.1);
  }
}

@keyframes popIn {
  0% {
    opacity: 0;
    transform: scale(0.8);
  }
  50% {
    opacity: 1;
    transform: scale(1.05);
  }
  100% {
    opacity: 1;
    transform: scale(1);
  }
}

@keyframes shimmer {
  0% {
    background-position: -1000px 0;
  }
  100% {
    background-position: 1000px 0;
  }
}

@keyframes pulse-glow {
  0%, 100% {
    box-shadow: 0 0 20px rgba(15, 139, 125, 0);
  }
  50% {
    box-shadow: 0 0 30px rgba(15, 139, 125, 0.3);
  }
}
```

```
/* Logs section styles */
.logs-list {
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
  max-height: 400px;
  overflow-y: auto;
  padding-right: 0.5rem;
}

.logs-list::-webkit-scrollbar {
  width: 6px;
}

.logs-list::-webkit-scrollbar-track {
  background: rgba(100, 90, 77, 0.06);
  border-radius: 10px;
}

.logs-list::-webkit-scrollbar-thumb {
  background: rgba(15, 118, 110, 0.3);
  border-radius: 10px;
}

.logs-list::-webkit-scrollbar-thumb:hover {
  background: rgba(15, 118, 110, 0.5);
}

.log-item {
  background: rgba(255, 255, 255, 0.6);
  border-left: 5px solid transparent;
  border-radius: 0.85rem;
  padding: 1.1rem;
  margin-bottom: 0;
  animation: slideUp 0.45s cubic-bezier(0.34, 1.56, 0.64, 1) forwards;
  box-shadow: var(--shadow-sm);
  transition: all 0.3s cubic-bezier(0.34, 1.56, 0.64, 1);
  position: relative;
  overflow: hidden;
}

.log-item::before {
  content: '';
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  height: 1px;
```

```
  background: linear-gradient(90deg, transparent, currentColor, transparent);
  opacity: 0.1;
}

.log-item:hover {
  box-shadow: var(--shadow-md);
  transform: translateX(6px);
  border-left-color: currentColor;
}

.log-item.log-success {
  border-left-color: var(--success);
  background: linear-gradient(135deg, rgba(16, 185, 129, 0.12), rgba(16, 185, 129,
0.05));
}

.log-item.log-error {
  border-left-color: var(--danger);
  background: linear-gradient(135deg, rgba(239, 68, 68, 0.12), rgba(239, 68, 68,
0.05));
}

.log-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 0.5rem;
  gap: 1rem;
}

.log-type {
  font-weight: 700;
  font-size: 0.95rem;
  color: var(--brand-deep);
  text-transform: uppercase;
  letter-spacing: 0.05em;
}

.log-time {
  font-size: 0.85rem;
  color: var(--muted);
  white-space: nowrap;
  background: rgba(15, 118, 110, 0.08);
  padding: 0.25rem 0.6rem;
  border-radius: 0.4rem;
}

.log-details {
  display: flex;
```

```
  flex-direction: column;
  gap: 0.35rem;
}

.log-details p {
  margin: 0;
  font-size: 0.9rem;
  line-height: 1.4;
}

.log-contact {
  color: var(--text);
}

.log-contact strong {
  color: var(--brand-deep);
  font-weight: 700;
}

.log-message {
  color: var(--muted);
  font-size: 0.88rem;
}

.log-item.log-success .log-message {
  color: #059669;
}

.log-item.log-error .log-message {
  color: #dc2626;
}

@keyframes slideUp {
  from {
    opacity: 0;
    transform: translateY(12px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

@media (max-width: 768px) {
  .logs-list {
    max-height: 300px;
  }

  .log-item {
```

```
    padding: 0.8rem;
  }

  .log-header {
    flex-direction: column;
    align-items: flex-start;
    margin-bottom: 0.4rem;
  }

  .log-time {
    align-self: flex-end;
    margin-top: -0.3rem;
  }
}
/* Enhanced section headers */
section h2 {
  font-weight: 800;
  letter-spacing: -0.5px;
}

.step-badge {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: 36px;
  height: 36px;
  border-radius: 50%;
  background: linear-gradient(135deg, var(--brand) 0%, var(--brand-light) 100%);
  color: #fff;
  font-weight: 800;
  font-size: 1.1rem;
  margin-right: 0.6rem;
  box-shadow: var(--shadow-md);
  transition: all 0.3s cubic-bezier(0.34, 1.56, 0.64, 1);
  display: inline-flex;
}

.step-badge:hover {
  transform: scale(1.1) rotate(5deg);
  box-shadow: var(--shadow-lg);
}

/* Better form labels */
.form-label {
  font-weight: 700;
  color: var(--text);
  font-size: 0.95rem;
  margin-bottom: 0.6rem;
  letter-spacing: 0.3px;
```

```
  position: relative;
  padding-left: 0;
}

.form-label::after {
  content: '';
  position: absolute;
  bottom: -4px;
  left: 0;
  width: 24px;
  height: 2px;
  background: linear-gradient(90deg, var(--brand), transparent);
  border-radius: 1px;
}

/* Enhanced output box */
.output-box {
  min-height: 350px;
  background: linear-gradient(135deg, var(--output-bg) 0%, #1a1515 100%);
  color: var(--output-text);
  border-radius: 1.1rem;
  border: 1px solid rgba(245, 158, 11, 0.25);
  padding: 1.25rem;
  overflow-x: auto;
  white-space: pre-wrap;
  word-break: break-word;
  font-family: 'Monaco', 'Menlo', 'Ubuntu Mono', monospace;
  font-size: 0.9rem;
  line-height: 1.5;
  box-shadow: inset 0 2px 8px rgba(0, 0, 0, 0.3);
  transition: all 0.3s ease;
}

.output-box:hover {
  border-color: rgba(245, 158, 11, 0.4);
}

/* Enhanced token pill */
.token-pill {
  background: linear-gradient(135deg, #fff 0%, rgba(255, 255, 255, 0.9) 100%);
  color: var(--brand-deep);
  border: 1.5px solid rgba(15, 139, 125, 0.25);
  padding: 0.7rem 1.2rem;
  border-radius: 999px;
  font-weight: 800;
  font-size: 0.95rem;
  white-space: nowrap;
  box-shadow: var(--shadow-sm);
  transition: all 0.3s ease;
```

```
  animation: popIn 0.6s cubic-bezier(0.34, 1.56, 0.64, 1) both;
}

.token-pill.is-authenticated {
  background: linear-gradient(135deg, rgba(16, 185, 129, 0.15), rgba(32, 201, 151,
0.1));
  border-color: var(--success);
  color: var(--success);
  box-shadow: 0 0 20px rgba(16, 185, 129, 0.2);
}

.token-pill.is-authenticated::after {
  content: ' âœ“';
  font-weight: 800;
}

/* Enhanced secondary buttons */
.btn-sm {
  font-weight: 700;
  transition: all 0.3s cubic-bezier(0.34, 1.56, 0.64, 1);
}

.btn-outline-secondary {
  border-color: rgba(107, 95, 83, 0.3);
  color: var(--muted);
}

.btn-outline-secondary:hover {
  background: rgba(107, 95, 83, 0.1);
  border-color: var(--muted);
  color: var(--text);
  transform: translateY(-2px);
  box-shadow: var(--shadow-sm);
}
/* OTP Input Enhancement */
.otp-input {
  letter-spacing: 0.3em;
  font-weight: 800;
  font-size: 1.5rem;
  text-align: center;
}

/* Responsive Media Queries */
@media (max-width: 768px) {
  .hero {
    border-left-width: 5px;
  }

  .output-box {
```

```
    min-height: 260px;
  }

  .step-badge {
    width: 32px;
    height: 32px;
    font-size: 0.95rem;
  }

  .card-soft {
    border-radius: 1rem;
  }

  .token-pill {
    padding: 0.6rem 1rem;
    font-size: 0.9rem;
  }

  .btn-brand {
    font-size: 0.95rem;
  }

  .logs-list {
    max-height: 300px;
  }
}

@media (max-width: 480px) {
  h1 {
    font-size: 1.75rem;
  }

  .form-control,
  .form-select,
  .btn {
    font-size: 1rem;
  }

  .output-box {
    padding: 0.75rem;
    min-height: 200px;
  }

  .log-item {
    padding: 0.85rem;
  }
}
.hero {
  border-left: 8px solid var(--brand);
```

```
  position: relative;
  overflow: hidden;
}

.hero::after {
  content: '';
  position: absolute;
  top: 0;
  right: 0;
  width: 200px;
  height: 200px;
  background: radial-gradient(circle, rgba(32, 201, 151, 0.1), transparent);
  border-radius: 50%;
  pointer-events: none;
}

.eyebrow {
  color: var(--brand-deep);
  text-transform: uppercase;
  letter-spacing: 0.1em;
  font-weight: 800;
  font-size: 0.8rem;
  position: relative;
  display: inline-block;
  padding-bottom: 0.5rem;
}

.eyebrow::after {
  content: '';
  position: absolute;
  bottom: 0;
  left: 0;
  width: 20px;
  height: 2px;
  background: linear-gradient(90deg, var(--brand), var(--brand-light));
  border-radius: 1px;
}

.subtitle {
  max-width: 58ch;
  color: var(--muted);
  line-height: 1.6;
  font-size: 1.05rem;
  font-weight: 500;
}
/* Container & spacing improvements */
main {
  animation: fadeIn 0.8s ease 0.2s both;
}
```

```
.row {
  animation: staggerChildren 0.8s ease-out both;
}

.row > [class*="col-"] {
  animation: riseIn 0.6s cubic-bezier(0.34, 1.56, 0.64, 1) both;
}

.row > [class*="col-"]:nth-child(1) { animation-delay: 0s; }
.row > [class*="col-"]:nth-child(2) { animation-delay: 0.1s; }
.row > [class*="col-"]:nth-child(3) { animation-delay: 0.2s; }
.row > [class*="col-"]:nth-child(4) { animation-delay: 0.3s; }
.row > [class*="col-"]:nth-child(5) { animation-delay: 0.4s; }

/* Form group improvements */
.form-group,
.mb-3 {
  animation: fadeIn 0.5s ease both;
}

/* Badge enhancements */
.badge {
  font-weight: 800;
  padding: 0.5rem 0.85rem;
  letter-spacing: 0.3px;
  border-radius: 0.6rem;
  transition: all 0.3s cubic-bezier(0.34, 1.56, 0.64, 1);
}

.badge:hover {
  transform: scale(1.05);
}

.badge.bg-brand {
  background: linear-gradient(135deg, var(--brand) 0%, var(--brand-light)
100%) !important;
  box-shadow: var(--shadow-sm);
}

/* Button group spacing */
.btn + .btn {
  margin-left: 0.5rem;
}

/* Enhanced focus visible */
:focus-visible {
  outline: 2px solid var(--brand);
  outline-offset: 2px;
```

```
  border-radius: 0.5rem;
}

/* Smooth text selection */
::selection {
  background: rgba(15, 139, 125, 0.3);
  color: var(--text);
}

::-moz-selection {
  background: rgba(15, 139, 125, 0.3);
  color: var(--text);
}

/* Better scrollbar styling */
::-webkit-scrollbar {
  width: 8px;
}

::-webkit-scrollbar-track {
  background: rgba(107, 95, 83, 0.05);
  border-radius: 10px;
}

::-webkit-scrollbar-thumb {
  background: rgba(15, 139, 125, 0.25);
  border-radius: 10px;
  transition: all 0.3s ease;
}

::-webkit-scrollbar-thumb:hover {
  background: rgba(15, 139, 125, 0.5);
}

/* Logs section */
.logs-list {
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
  max-height: 400px;
  overflow-y: auto;
  padding-right: 0.5rem;
}
@keyframes pulse-glow {
  0%, 100% {
    box-shadow: 0 0 20px rgba(15, 139, 125, 0);
  }
  50% {
    box-shadow: 0 0 30px rgba(15, 139, 125, 0.3);
```

```
  }
}

@keyframes fadeIn {
  from {
    opacity: 0;
  }
  to {
    opacity: 1;
  }
}

@keyframes staggerChildren {
  0% {
    opacity: 0;
  }
  100% {
    opacity: 1;
  }
}

@keyframes slideUp {
  from {
    opacity: 0;
    transform: translateY(16px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}
.btn-outline-secondary:hover {
  background: rgba(107, 95, 83, 0.1);
  border-color: var(--muted);
  color: var(--text);
  transform: translateY(-2px);
  box-shadow: var(--shadow-sm);
}

/* Bootstrap success button override */
.btn-success {
  background: linear-gradient(135deg, var(--success) 0%, #059669 100%);
  border: none;
  font-weight: 700;
  font-size: 1rem;
  letter-spacing: 0.02em;
  box-shadow: var(--shadow-md);
  transition: all 0.3s cubic-bezier(0.34, 1.56, 0.64, 1);
  position: relative;
```

```
  overflow: hidden;
}

.btn-success::before {
  content: '';
  position: absolute;
  top: 50%;
  left: 50%;
  width: 0;
  height: 0;
  border-radius: 50%;
  background: rgba(255, 255, 255, 0.3);
  transform: translate(-50%, -50%);
  transition: width 0.6s, height 0.6s;
}

.btn-success:hover::before {
  width: 300px;
  height: 300px;
}

.btn-success:hover,
.btn-success:focus {
  background: linear-gradient(135deg, #059669 0%, var(--success) 100%);
  box-shadow: 0 12px 32px rgba(16, 185, 129, 0.35);
  transform: translateY(-3px);
  color: #fff;
}

/* Button group enhancements */
.btn-group > .btn,
.btn-group-vertical > .btn {
  transition: all 0.3s cubic-bezier(0.34, 1.56, 0.64, 1);
}

/* Better button sizing */
.btn-lg {
  padding: 0.75rem 1.5rem;
  font-size: 1.05rem;
}

.btn-sm {
  padding: 0.4rem 0.8rem;
  font-size: 0.9rem;
}
~~~

### File Name: frontend/script.js
Purpose: Handles API calls, logs rendering, auth token state, and form interactions.
```

~~~js
const API_BASE = 'http://localhost:5000/api';
let authToken = '';
let verificationLogs = [];

const output = document.getElementById('output');
const tokenStatus = document.getElementById('tokenStatus');
const logsContainer = document.getElementById('logsContainer');
const logCount = document.getElementById('logCount');

const setTokenStatus = (isAuthenticated) => {
  if (!tokenStatus) return;

  if (isAuthenticated) {
    tokenStatus.textContent = 'Token: Verified';
    tokenStatus.classList.add('is-authenticated');
    return;
  }

  tokenStatus.textContent = 'Token: Not Verified';
  tokenStatus.classList.remove('is-authenticated');
};

const showOutput = (data) => {
  output.textContent = typeof data === 'string' ? data : JSON.stringify(data, null, 2);
};

const addLog = (type, contact, details, status = 'success') => {
  const timestamp = new Date().toLocaleTimeString('en-US', {
    hour: '2-digit',
    minute: '2-digit',
    second: '2-digit',
    hour12: true,
  });

  verificationLogs.unshift({
    id: Date.now(),
    type,
    contact,
    details,
    status,
    timestamp,
  });

  renderLogs();
};

const renderLogs = () => {
  if (verificationLogs.length === 0) {
~~~

```
    logsContainer.innerHTML = '<p class="text-secondary text-center py-4">No
verifications yet. Start by generating an OTP.</p>';
    logCount.textContent = '0';
    return;
  }

  logCount.textContent = verificationLogs.length;

  logsContainer.innerHTML = verificationLogs
    .map((log) => {
      const typeEmoji = {
        'otp-generated': 'ðŸ"¤',
        'otp-verified': 'â œ…',
        'user-created': 'ðŸ'¤',
        error: 'â Œ',
      }[log.type] || 'ðŸ" ';

      const statusClass = log.status === 'success' ? 'log-success' : 'log-error';

      return `
        <div class="log-item ${statusClass}">
          <div class="log-header">
            <span class="log-type">${typeEmoji} ${log.type.replace('-', '
').toUpperCase()}</span>
            <span class="log-time">${log.timestamp}</span>
          </div>
          <div class="log-details">
            <p class="log-contact"><strong>Contact:</strong> ${log.contact}</p>
            <p class="log-message">${log.details}</p>
          </div>
        </div>
      `;
    })
    .join('');
};

const request = async (url, options = {}) => {
  const response = await fetch(url, options);
  const data = await response.json();

  if (!response.ok) {
    throw new Error(data.message || 'Request failed');
  }

  return data;
};

document.getElementById('generateForm').addEventListener('submit', async (event) => {
  event.preventDefault();
```

```
  const generateForm = document.getElementById('generateForm');
  const contact = document.getElementById('contact').value;

  try {
    const data = await request(`${API_BASE}/auth/generate-otp`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ contact }),
    });

    showOutput(data);
    addLog('otp-generated', contact, `OTP sent successfully. Expires in 5 minutes.`);
    generateForm.reset();
    document.getElementById('verifyContact').value = data?.data?.contact || contact;
  } catch (error) {
    showOutput(error.message);
    addLog('otp-generated', contact, `Failed: ${error.message}`, 'error');
  }
});

document.getElementById('verifyForm').addEventListener('submit', async (event) => {
  event.preventDefault();

  const verifyForm = document.getElementById('verifyForm');

  const contact = document.getElementById('verifyContact').value;
  const otp = document.getElementById('otp').value;

  try {
    const data = await request(`${API_BASE}/auth/verify-otp`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ contact, otp }),
    });

    authToken = data.token;
    setTokenStatus(true);
    showOutput({ ...data, info: 'Token stored in memory for protected APIs.' });
    addLog('otp-verified', contact, `Successfully verified. Token received.`);
    verifyForm.reset();
  } catch (error) {
    showOutput(error.message);
    addLog('otp-verified', contact, `Verification failed: ${error.message}`, 'error');
  }
});

document.getElementById('fetchUsersBtn').addEventListener('click', async () => {
  try {
    const data = await request(`${API_BASE}/users`, {
```

```
      method: 'GET',
      headers: {
        Authorization: `Bearer ${authToken}`,
      },
    });

    showOutput(data);
  } catch (error) {
    showOutput(error.message);
  }
});

document.getElementById('createUserForm').addEventListener('submit', async (event) =>
{
  event.preventDefault();

  const payload = {
    name: document.getElementById('name').value,
    contact: document.getElementById('userContact').value,
    contactType: document.getElementById('contactType').value,
  };

  try {
    const data = await request(`${API_BASE}/users`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        Authorization: `Bearer ${authToken}`,
      },
      body: JSON.stringify(payload),
    });

    showOutput(data);
    addLog('user-created', payload.contact, `User "${payload.name}" created
successfully.`);
    document.getElementById('createUserForm').reset();
  } catch (error) {
    showOutput(error.message);
    addLog('user-created', payload.contact, `Failed: ${error.message}`, 'error');
  }
});

document.getElementById('clearLogsBtn').addEventListener('click', () => {
  verificationLogs = [];
  renderLogs();
});
~~~
```

4. paste your project UI or output (Screenshots etc)

- Frontend Page Slot 1
- Frontend Page Slot 2
- Everything Else Slot (Combined)

Everything Else Slot (Combined)
- [ ] Frontend screenshot item: Frontend page loaded
- [ ] Generate OTP success response
- [ ] Verify OTP success response
- [ ] CRUD list users response
- [ ] Error case (invalid/expired OTP)
- [ ] Backend Server Side Responses
- [ ] Data in Mongo Compass/Shell
- [ ] Postman otp verification.

5. Complete README Content (Verbatim)

OTP Verification System Project Documentation

1. Overview

This project is a full-stack OTP Verification System that verifies users through one-time passwords sent to email or phone. It combines a modern frontend dashboard with a Node.js and Express backend that supports OTP generation, OTP verification, expiry handling, and JWT-based access for protected APIs. The system also includes CRUD operations for users, where only OTP-verified contacts are allowed.

The project is designed with modular backend architecture, clean middleware layering, and practical real-world notification support using Nodemailer and Twilio. Persistence is handled through MongoDB with Mongoose models for both users and OTPs, so data created from the web app is stored in the database rather than temporary memory.

2. Tech Stack : Programming Languages, Scripting Languages, and Tools Used

- JavaScript (Node.js): Used for backend API, business logic, and frontend scripting.
- HTML5: Used to build the structure of the frontend pages and forms.
- CSS3: Used for styling, responsive UI design, and animation effects.
- Node.js: Runtime for executing backend JavaScript code.
- Express.js: Framework used to build REST APIs.
- MongoDB: Database used to persist users and OTP records.
- Mongoose: ODM used for defining schemas and interacting with MongoDB.
- JSON Web Token (jsonwebtoken): Used for authentication and protected routes.
- express-validator: Used for validating incoming API request payloads.
- Nodemailer: Used to send OTP over email.
- Twilio: Used to send OTP over SMS.
- Morgan: HTTP request logger middleware.
- CORS: Enables secure cross-origin API requests.
- dotenv: Loads environment variables from .env file.

- nodemon: Restarts backend automatically during development.

3. Full Backend, Frontend & DB codes with File Name and Purpose

File Name: backend/package.json
Purpose: Backend package manifest containing scripts and dependencies.
~~~json
{
  "name": "otp-verification-backend",
  "version": "1.0.0",
  "description": "OTP verification system backend using Node.js and Express",
  "main": "src/server.js",
  "scripts": {
    "start": "node src/server.js",
    "dev": "nodemon src/server.js"
  },
  "keywords": [
    "otp",
    "express",
    "nodejs",
    "mongodb"
  ],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "express-validator": "^7.2.0",
    "jsonwebtoken": "^9.0.2",
    "mongoose": "^8.8.1",
    "morgan": "^1.10.0",
    "nodemailer": "^6.10.0",
    "twilio": "^5.4.2"
  },
  "devDependencies": {
    "nodemon": "^3.1.7"
  }
}
~~~

File Name: backend/.env.example
Purpose: Environment variable template required to run the backend.
~~~env
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/otp_verification_system
JWT_SECRET=change_this_secret
OTP_EXPIRY_MINUTES=5
~~~

```
# Nodemailer SMTP config (for email OTP)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your_email@gmail.com
SMTP_PASS=your_email_app_password
SMTP_FROM=your_email@gmail.com

# Twilio config (required for real SMS OTP delivery)
# Use E.164 format for TWILIO_PHONE_NUMBER, for example: +14155552671
TWILIO_ACCOUNT_SID=your_twilio_account_sid
TWILIO_AUTH_TOKEN=your_twilio_auth_token
TWILIO_PHONE_NUMBER=+12345678901
SMS_DEFAULT_COUNTRY_CODE=+91
~~~


### File Name: backend/src/config/env.js
Purpose: Reads and exports application environment configuration.
~~~js
const dotenv = require('dotenv');

dotenv.config();

module.exports = {
  port: process.env.PORT || 5000,
  mongoUri: process.env.MONGO_URI || '',
  jwtSecret: process.env.JWT_SECRET || 'dev-secret-key',
  otpExpiryMinutes: Number(process.env.OTP_EXPIRY_MINUTES || 5),
  smtpHost: process.env.SMTP_HOST || '',
  smtpPort: Number(process.env.SMTP_PORT || 587),
  smtpUser: process.env.SMTP_USER || '',
  smtpPass: process.env.SMTP_PASS || '',
  smtpFrom: process.env.SMTP_FROM || process.env.SMTP_USER || '',
  twilioAccountSid: process.env.TWILIO_ACCOUNT_SID || '',
  twilioAuthToken: process.env.TWILIO_AUTH_TOKEN || '',
  twilioPhoneNumber: process.env.TWILIO_PHONE_NUMBER || '',
  smsDefaultCountryCode: process.env.SMS_DEFAULT_COUNTRY_CODE || '+91',
};
~~~


### File Name: backend/src/config/db.js //Database Connection.
Purpose: Connects the backend to MongoDB through Mongoose.
~~~js
const mongoose = require('mongoose');
const env = require('./env');

const connectDB = async () => {
  if (!env.mongoUri) {
    throw new Error('MONGO_URI is not set');
```

```
  }

  await mongoose.connect(env.mongoUri);
  console.log('MongoDB connected successfully.');
};

module.exports = { connectDB };
```

File Name: backend/src/app.js
Purpose: Creates Express app, adds middleware, and mounts routes.
```js
const express = require('express');
const cors = require('cors');
const morgan = require('morgan');

const authRoutes = require('./routes/authRoutes');
const userRoutes = require('./routes/userRoutes');
const { notFoundHandler, errorHandler } = require('./middlewares/errorMiddleware');

const app = express();

app.use(cors());
app.use(express.json());
app.use(morgan('dev'));

app.get('/api/health', (req, res) => {
  res.status(200).json({ success: true, message: 'Server is running' });
});

app.use('/api/auth', authRoutes);
app.use('/api/users', userRoutes);

app.use(notFoundHandler);
app.use(errorHandler);

module.exports = app;
```

File Name: backend/src/server.js
Purpose: Starts the backend server after DB connection.
```js
const app = require('./app');
const env = require('./config/env');
const { connectDB } = require('./config/db');

const startServer = async () => {
  try {
    await connectDB();
```

```
    app.listen(env.port, () => {
      console.log(`Server listening on port ${env.port}`);
    });
  } catch (error) {
    console.error('Failed to start server:', error.message);
    process.exit(1);
  }
};

startServer();
~~~

### File Name: backend/src/models/User.js
Purpose: Defines User schema and model.
~~~js
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true,
    },
    contact: {
      type: String,
      required: true,
      unique: true,
      trim: true,
    },
    contactType: {
      type: String,
      enum: ['email', 'phone'],
      required: true,
    },
    isVerified: {
      type: Boolean,
      default: false,
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model('User', userSchema);
~~~

### File Name: backend/src/models/Otp.js
Purpose: Defines OTP schema and model.
```

~~~js
const mongoose = require('mongoose');

const otpSchema = new mongoose.Schema(
  {
    contact: {
      type: String,
      required: true,
      trim: true,
      index: true,
    },
    otp: {
      type: String,
      required: true,
    },
    expiresAt: {
      type: Date,
      required: true,
    },
    isUsed: {
      type: Boolean,
      default: false,
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model('Otp', otpSchema);
~~~

File Name: backend/src/utils/ApiError.js
Purpose: Standardized API error class.
~~~js
class ApiError extends Error {
  constructor(statusCode, message) {
    super(message);
    this.statusCode = statusCode;
  }
}

module.exports = ApiError;
~~~

File Name: backend/src/utils/generateOtp.js
Purpose: Generates 6-digit numeric OTP.
~~~js
const generateOtp = () => {
  return Math.floor(100000 + Math.random() * 900000).toString();
};
~~~

```
module.exports = generateOtp;
~~~

### File Name: backend/src/middlewares/authMiddleware.js
Purpose: Validates JWT token for protected routes.
~~~js
const jwt = require('jsonwebtoken');
const env = require('../config/env');
const ApiError = require('../utils/ApiError');

const protect = (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return next(new ApiError(401, 'Authentication token missing'));
  }

  const token = authHeader.split(' ')[1];

  try {
    const decoded = jwt.verify(token, env.jwtSecret);
    req.user = decoded;
    next();
  } catch (error) {
    next(new ApiError(401, 'Invalid or expired token'));
  }
};

module.exports = { protect };
~~~

### File Name: backend/src/middlewares/validateMiddleware.js
Purpose: Formats and returns validation errors from express-validator.
~~~js
const { validationResult } = require('express-validator');

const validateRequest = (req, res, next) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({
      success: false,
      message: 'Validation failed',
      errors: errors.array(),
    });
  }
  next();
};
```

```
module.exports = { validateRequest };
~~~

### File Name: backend/src/middlewares/errorMiddleware.js
Purpose: Handles unknown routes and global API errors.
~~~js
const ApiError = require('../utils/ApiError');

const notFoundHandler = (req, res, next) => {
  next(new ApiError(404, `Route not found: ${req.originalUrl}`));
};

const errorHandler = (err, req, res, next) => {
  const statusCode = err.statusCode || 500;
  const message = err.message || 'Internal server error';

  res.status(statusCode).json({
    success: false,
    message,
  });
};

module.exports = { notFoundHandler, errorHandler };
~~~

### File Name: backend/src/validators/authValidators.js
Purpose: Validates generate and verify OTP request payloads.
~~~js
const { body } = require('express-validator');

const contactValidator = body('contact')
  .notEmpty()
  .withMessage('Contact is required')
  .isString()
  .withMessage('Contact must be a string')
  .customSanitizer((value) => String(value || '').trim())
  .custom((value) => {
    const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    const phonePattern = /^\+?[0-9\s()-]{10,18}$/;

    if (emailPattern.test(value) || phonePattern.test(value)) {
      return true;
    }

    throw new Error('Contact must be a valid email or phone number');
  });

const otpValidator = body('otp')
  .notEmpty()
```

```
  .withMessage('OTP is required')
  .matches(/^\d{6}$/)
  .withMessage('OTP must be a 6-digit number');

const generateOtpValidation = [contactValidator];
const verifyOtpValidation = [contactValidator, otpValidator];

module.exports = { generateOtpValidation, verifyOtpValidation };
~~~


### File Name: backend/src/validators/userValidators.js
Purpose: Validates user CRUD request fields and params.
~~~js
const { body, param } = require('express-validator');

const contactTypeValidator = body('contactType')
  .notEmpty()
  .withMessage('contactType is required')
  .isIn(['email', 'phone'])
  .withMessage('contactType must be email or phone');

const createUserValidation = [
  body('name').notEmpty().withMessage('Name is required').isString().trim(),
  body('contact').notEmpty().withMessage('Contact is required').isString().trim(),
  contactTypeValidator,
];

const updateUserValidation = [
  param('id').notEmpty().withMessage('User ID is required'),
  body('name').optional().isString().withMessage('Name must be a string').trim(),
  body('contact').optional().isString().withMessage('Contact must be a string').trim(),
  body('contactType').optional().isIn(['email', 'phone']).withMessage('contactType
must be email or phone'),
];

const userIdValidation = [param('id').notEmpty().withMessage('User ID is required')];

module.exports = { createUserValidation, updateUserValidation, userIdValidation };
~~~


### File Name: backend/src/services/otpService.js
Purpose: Handles OTP creation, lookup, usage marking, and expiry cleanup.
~~~js
const { useMongo } = require('../config/db');
const Otp = require('../models/Otp');
const { memoryOtps } = require('../data/memoryStore');

const createOtpEntry = async ({ contact, otp, expiresAt }) => {
  if (useMongo()) {
```

```
    await Otp.deleteMany({ contact, isUsed: false });
    return Otp.create({ contact, otp, expiresAt, isUsed: false });
  }

  for (let i = memoryOtps.length - 1; i >= 0; i -= 1) {
    if (memoryOtps[i].contact === contact && !memoryOtps[i].isUsed) {
      memoryOtps.splice(i, 1);
    }
  }

  const otpEntry = {
    id: Date.now().toString(),
    contact,
    otp,
    expiresAt,
    isUsed: false,
  };
  memoryOtps.push(otpEntry);
  return otpEntry;
};

const getLatestUnusedOtpByContact = async (contact) => {
  if (useMongo()) {
    return Otp.findOne({ contact, isUsed: false }).sort({ createdAt: -1 });
  }

  const filtered = memoryOtps
    .filter((item) => item.contact === contact && !item.isUsed)
    .sort((a, b) => new Date(b.expiresAt) - new Date(a.expiresAt));
  return filtered[0] || null;
};

const markOtpAsUsed = async (otpEntry) => {
  if (!otpEntry) return;

  if (useMongo()) {
    otpEntry.isUsed = true;
    await otpEntry.save();
    return;
  }

  otpEntry.isUsed = true;
};

const cleanupExpiredOtps = async () => {
  const now = new Date();

  if (useMongo()) {
    await Otp.deleteMany({ expiresAt: { $lt: now } });
```

```
    return;
  }

  for (let i = memoryOtps.length - 1; i >= 0; i -= 1) {
    if (new Date(memoryOtps[i].expiresAt) < now) {
      memoryOtps.splice(i, 1);
    }
  }
};

module.exports = {
  createOtpEntry,
  getLatestUnusedOtpByContact,
  markOtpAsUsed,
  cleanupExpiredOtps,
};
~~~


### File Name: backend/src/services/notificationService.js
Purpose: Sends OTP notifications through email (Nodemailer) or SMS (Twilio).
~~~js
const nodemailer = require('nodemailer');
const twilio = require('twilio');

const env = require('../config/env');

let emailTransporter;
let smsClient;

const getEmailTransporter = () => {
  if (emailTransporter) return emailTransporter;

  if (!env.smtpHost || !env.smtpPort || !env.smtpUser || !env.smtpPass
|| !env.smtpFrom) {
    return null;
  }

  emailTransporter = nodemailer.createTransport({
    host: env.smtpHost,
    port: env.smtpPort,
    secure: env.smtpPort === 465,
    auth: {
      user: env.smtpUser,
      pass: env.smtpPass,
    },
  });

  return emailTransporter;
};
```

```
const getSmsClient = () => {
  if (smsClient) return smsClient;

  if (!env.twilioAccountSid || !env.twilioAuthToken || !env.twilioPhoneNumber) {
    return null;
  }

  smsClient = twilio(env.twilioAccountSid, env.twilioAuthToken);
  return smsClient;
};

const normalizePhoneNumber = (contact) => {
  if (contact.startsWith('+')) return contact;
  return `${env.smsDefaultCountryCode}${contact}`;
};

const buildOtpMessage = (otp, expiresAt) => {
  const expiresAtText = new Date(expiresAt).toLocaleString();
  return [
    'OTP Verification Code',
    '',
    `Your one-time password is: ${otp}`,
    `This code expires at: ${expiresAtText}`,
    '',
    'If you did not request this code, please ignore this message.',
    'Do not share this OTP with anyone.',
  ].join('\n');
};

const buildOtpEmailHtml = ({ otp, expiresAt, contact }) => {
  const expiresAtText = new Date(expiresAt).toLocaleString();

  return `
  <!DOCTYPE html>
  <html lang="en">
    <head>
      <meta charset="UTF-8" />
      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
      <title>OTP Verification Code</title>
    </head>
    <body style="margin:0;padding:0;background:#f4f6fb;font-family:Segoe
UI,Arial,sans-serif;color:#1f2937;">
      <table role="presentation" width="100%" cellspacing="0" cellpadding="0"
border="0" style="background:#f4f6fb;padding:24px 12px;">
        <tr>
          <td align="center">
            <table role="presentation" width="100%" cellspacing="0" cellpadding="0"
border="0" style="max-width:620px;background:#ffffff;border-
```

```
radius:16px;overflow:hidden;border:1px solid #e5e7eb;box-shadow:0 12px 36px
rgba(15,23,42,0.12);">
              <tr>
                <td style="padding:28px 32px;background:linear-
gradient(135deg,#0f8b7d,#14b8a6);color:#ffffff;">
                  <p style="margin:0;font-size:12px;letter-spacing:1.4px;text-
transform:uppercase;opacity:0.9;">Secure Verification</p>
                  <h1 style="margin:8px 0 0;font-size:24px;line-height:1.2;font-
weight:700;">Your OTP Code</h1>
                </td>
              </tr>

              <tr>
                <td style="padding:30px 32px 10px;">
                  <p style="margin:0 0 14px;font-size:15px;line-height:1.65;">Hi,</p>
                  <p style="margin:0 0 22px;font-size:15px;line-height:1.65;">
                    Use the verification code below to complete your login or account
action for
                    <strong>${contact}</strong>.
                  </p>

                  <table role="presentation" width="100%" cellspacing="0"
cellpadding="0" border="0" style="margin:0 0 22px;">
                    <tr>
                      <td align="center" style="padding:20px;border:1px solid
#ccfbf1;background:#f0fdfa;border-radius:14px;">
                        <p style="margin:0 0 8px;font-size:12px;color:#0f766e;text-
transform:uppercase;letter-spacing:1.2px;font-weight:700;">One-Time Password</p>
                        <p style="margin:0;font-size:36px;line-height:1;letter-
spacing:10px;font-weight:800;color:#0f172a;">${otp}</p>
                      </td>
                    </tr>
                  </table>

                  <table role="presentation" width="100%" cellspacing="0"
cellpadding="0" border="0" style="margin:0 0 20px;">
                    <tr>
                      <td style="padding:14px 16px;background:#fff7ed;border:1px solid
#fed7aa;border-radius:10px;font-size:13px;line-height:1.5;color:#9a3412;">
                        This OTP is valid until <strong>${expiresAtText}</strong>.
                      </td>
                    </tr>
                  </table>

                  <p style="margin:0 0 10px;font-size:13px;line-
height:1.6;color:#6b7280;">
                    Security tip: Never share this OTP with anyone, including support
staff.
                  </p>
```

```
              </td>
            </tr>

            <tr>
              <td style="padding:18px 32px 26px;border-top:1px solid #f1f5f9;">
                <p style="margin:0;font-size:12px;line-height:1.6;color:#94a3b8;">
                  If you did not request this code, you can safely ignore this email.
                </p>
              </td>
            </tr>
          </table>
        </td>
      </tr>
    </table>
  </body>
</html>
`;
};

const sendEmailOtp = async ({ contact, otp, expiresAt }) => {
  const transporter = getEmailTransporter();

  if (!transporter) {
    console.warn('SMTP is not fully configured. OTP will be logged instead of
emailed.');
    console.log(`OTP for ${contact}: ${otp} (expires at ${new
Date(expiresAt).toISOString()})`);
    return;
  }

  const message = buildOtpMessage(otp, expiresAt);
  const htmlMessage = buildOtpEmailHtml({ otp, expiresAt, contact });

  await transporter.sendMail({
    from: `OTP Verification System <${env.smtpFrom}>`,
    to: contact,
    subject: 'Your Premium OTP Verification Code',
    text: message,
    html: htmlMessage,
  });
};

const sendSmsOtp = async ({ contact, otp, expiresAt }) => {
  const client = getSmsClient();

  if (!client) {
    throw new Error(
      'SMS delivery is not configured. Please set TWILIO_ACCOUNT_SID,
TWILIO_AUTH_TOKEN, and TWILIO_PHONE_NUMBER in backend/.env',
```

```
    );
  }

  const to = normalizePhoneNumber(contact);
  const message = buildOtpMessage(otp, expiresAt);

  try {
    await client.messages.create({
      body: message,
      from: env.twilioPhoneNumber,
      to,
    });
  } catch (error) {
    throw new Error(`Failed to send SMS OTP: ${error.message}`);
  }
};

const sendOtpNotification = async ({ contact, contactType, otp, expiresAt }) => {
  if (contactType === 'email') {
    await sendEmailOtp({ contact, otp, expiresAt });
    return;
  }

  if (contactType === 'phone') {
    await sendSmsOtp({ contact, otp, expiresAt });
    return;
  }

  throw new Error('Unsupported contact type for OTP delivery');
};

module.exports = {
  sendOtpNotification,
};
~~~


### File Name: backend/src/controllers/authController.js
Purpose: Contains OTP generation and OTP verification controller actions.
~~~js
const jwt = require('jsonwebtoken');

const env = require('../config/env');
const User = require('../models/User');
const generateOtp = require('../utils/generateOtp');
const ApiError = require('../utils/ApiError');
const { sendOtpNotification } = require('../services/notificationService');
const {
  createOtpEntry,
  getLatestUnusedOtpByContact,
```

```
  markOtpAsUsed,
  cleanupExpiredOtps,
} = require('../services/otpService');

const normalizePhoneContact = (value) => {
  const compact = value.replace(/[\s()-]/g, '');

  if (compact.startsWith('+')) return compact;

  if (/^\d{10}$/.test(compact)) {
    return `${env.smsDefaultCountryCode}${compact}`;
  }

  return compact;
};

const normalizeContact = (contact) => {
  const trimmed = String(contact || '').trim();

  if (trimmed.includes('@')) {
    return trimmed.toLowerCase();
  }

  return normalizePhoneContact(trimmed);
};

const detectContactType = (contact) => {
  const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  const phonePattern = /^\+[0-9]{10,15}$|^[0-9]{10,15}$/;

  if (emailPattern.test(contact)) return 'email';
  if (phonePattern.test(contact)) return 'phone';
  return null;
};

const findUserByContact = async (contact) => {
  return User.findOne({ contact });
};

const generateOtpHandler = async (req, res, next) => {
  try {
    const contact = normalizeContact(req.body.contact);
    const contactType = detectContactType(contact);

    if (!contactType) {
      return next(new ApiError(400, 'Please provide a valid email or phone number'));
    }

    let user = await findUserByContact(contact);
```

```
    if (!user) {
      user = await User.create({
        name: 'OTP User',
        contact,
        contactType,
        isVerified: false,
      });
    }

    const otp = generateOtp();
    const expiresAt = new Date(Date.now() + env.otpExpiryMinutes * 60 * 1000);

    await createOtpEntry({ contact, otp, expiresAt });
    await sendOtpNotification({ contact, contactType, otp, expiresAt });

    res.status(200).json({
      success: true,
      message: 'OTP generated and sent successfully',
      data: {
        contact,
        expiresAt,
        // Exposing OTP for demo/testing. Remove this in production.
        otp,
      },
    });
  } catch (error) {
    next(error);
  }
};

const verifyOtpHandler = async (req, res, next) => {
  try {
    await cleanupExpiredOtps();

    const contact = normalizeContact(req.body.contact);
    const { otp } = req.body;
    const otpEntry = await getLatestUnusedOtpByContact(contact);

    if (!otpEntry) {
      return next(new ApiError(400, 'No active OTP found. Please generate OTP
again.'));
    }

    if (new Date(otpEntry.expiresAt) < new Date()) {
      return next(new ApiError(400, 'OTP expired. Please generate a new OTP.'));
    }

    if (otpEntry.otp !== otp) {
```

```
      return next(new ApiError(400, 'Invalid OTP'));
    }

    await markOtpAsUsed(otpEntry);

    let user = await findUserByContact(contact);

    if (user) {
      user.isVerified = true;
      await user.save();
    }

    const token = jwt.sign({ contact }, env.jwtSecret, { expiresIn: '1h' });

    res.status(200).json({
      success: true,
      message: 'OTP verified. Access granted.',
      token,
    });
  } catch (error) {
    next(error);
  }
};

module.exports = {
  generateOtpHandler,
  verifyOtpHandler,
};
~~~


### File Name: backend/src/controllers/userController.js
Purpose: Handles protected CRUD operations for verified users.
~~~js
const User = require('../models/User');
const ApiError = require('../utils/ApiError');

const listUsers = async (req, res, next) => {
  try {
    const users = await User.find({ isVerified: true }).sort({ createdAt: -1 });
    return res.status(200).json({ success: true, data: users });
  } catch (error) {
    next(error);
  }
};

const getUserById = async (req, res, next) => {
  try {
    const { id } = req.params;
```

```
    const user = await User.findById(id);
    if (!user) return next(new ApiError(404, 'User not found'));
    if (!user.isVerified) return next(new ApiError(403, 'Only verified users are
allowed in CRUD'));

    return res.status(200).json({ success: true, data: user });
  } catch (error) {
    next(error);
  }
};

const createUser = async (req, res, next) => {
  try {
    const { name, contact, contactType } = req.body;

    const existingUser = await User.findOne({ contact });
    if (!existingUser || !existingUser.isVerified) {
      return next(new ApiError(403, 'Contact must be OTP-verified before it can be
added to CRUD'));
    }

    if (existingUser.name !== 'OTP User') {
      return next(new ApiError(400, 'Contact already exists. Use update API to modify
user details'));
    }

    existingUser.name = name;
    existingUser.contactType = contactType;
    await existingUser.save();

    return res.status(201).json({
      success: true,
      message: 'Verified user added to CRUD successfully',
      data: existingUser,
    });
  } catch (error) {
    next(error);
  }
};

const updateUser = async (req, res, next) => {
  try {
    const { id } = req.params;

    const user = await User.findById(id);
    if (!user) return next(new ApiError(404, 'User not found'));
    if (!user.isVerified) return next(new ApiError(403, 'Only verified users are
allowed in CRUD'));
```

```
    Object.assign(user, req.body);
    const updatedUser = await user.save();

    return res.status(200).json({ success: true, message: 'User updated', data:
updatedUser });
  } catch (error) {
    next(error);
  }
};

const deleteUser = async (req, res, next) => {
  try {
    const { id } = req.params;

    const user = await User.findById(id);
    if (!user) return next(new ApiError(404, 'User not found'));
    if (!user.isVerified) return next(new ApiError(403, 'Only verified users are
allowed in CRUD'));

    await user.deleteOne();

    return res.status(200).json({ success: true, message: 'User deleted' });
  } catch (error) {
    next(error);
  }
};

module.exports = {
  listUsers,
  getUserById,
  createUser,
  updateUser,
  deleteUser,
};
~~~


### File Name: backend/src/routes/authRoutes.js
Purpose: Registers OTP auth routes.
~~~js
const express = require('express');

const { generateOtpHandler, verifyOtpHandler } =
require('../controllers/authController');
const { validateRequest } = require('../middlewares/validateMiddleware');
const { generateOtpValidation, verifyOtpValidation } =
require('../validators/authValidators');

const router = express.Router();
```

```
router.post('/generate-otp', generateOtpValidation, validateRequest,
generateOtpHandler);
router.post('/verify-otp', verifyOtpValidation, validateRequest, verifyOtpHandler);

module.exports = router;
~~~

### File Name: backend/src/routes/userRoutes.js
Purpose: Registers protected user CRUD routes.
~~~js
const express = require('express');

const {
  listUsers,
  getUserById,
  createUser,
  updateUser,
  deleteUser,
} = require('../controllers/userController');
const { protect } = require('../middlewares/authMiddleware');
const { validateRequest } = require('../middlewares/validateMiddleware');
const { createUserValidation, updateUserValidation, userIdValidation } =
require('../validators/userValidators');

const router = express.Router();

router.use(protect);

router.get('/', listUsers);
router.get('/:id', userIdValidation, validateRequest, getUserById);
router.post('/', createUserValidation, validateRequest, createUser);
router.put('/:id', updateUserValidation, validateRequest, updateUser);
router.delete('/:id', userIdValidation, validateRequest, deleteUser);

module.exports = router;
~~~

### File Name: frontend/index.html
Purpose: Builds the UI layout and form components for OTP flow and CRUD actions.
~~~html
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>OTP Verification System</title>
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />
    <link
```

```
href="https://fonts.googleapis.com/css2?family=Manrope:wght@400;600;700;800&display=swap"
      rel="stylesheet"
    />
    <link
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      rel="stylesheet"
      integrity="sha384-QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALEwIH"
      crossorigin="anonymous"
    />
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>
    <div class="bg-orb bg-orb-1" aria-hidden="true"></div>
    <div class="bg-orb bg-orb-2" aria-hidden="true"></div>

    <main class="container py-4 py-md-5">
      <section class="hero card-soft p-4 p-lg-5 mb-4">
        <div class="d-flex flex-column flex-lg-row gap-3 align-items-lg-center justify-content-between">
          <div>
            <p class="eyebrow mb-2">Secure Verification Flow</p>
            <h1 class="display-6 fw-bold mb-2">OTP Verification System</h1>
            <p class="subtitle mb-0">
              Generate OTP, verify users, and call protected CRUD APIs in one elegant dashboard.
            </p>
          </div>
          <div class="token-pill" id="tokenStatus">Token: Not Verified</div>
        </div>
      </section>

      <div class="row g-4">
        <div class="col-12 col-lg-6">
          <section class="card-soft p-4 h-100">
            <h2 class="h4 mb-3">
              <span class="step-badge">1</span> Generate OTP
            </h2>
            <form id="generateForm" class="vstack gap-3">
              <div>
                <label class="form-label" for="contact">Email or Phone</label>
                <input
                  class="form-control form-control-lg"
                  id="contact"
                  name="contact"
                  type="text"
                  inputmode="text"
```

```
          autocomplete="username"
          placeholder="example@mail.com or +911234567890"
          required
        />
      </div>
      <button class="btn btn-brand btn-lg w-100" type="submit">
        <span class="spinner-icon">âš¡</span> Generate OTP
      </button>
    </form>
  </section>
</div>

<div class="col-12 col-lg-6">
  <section class="card-soft p-4 h-100">
    <h2 class="h4 mb-3">
      <span class="step-badge">2</span> Verify OTP
    </h2>
    <form id="verifyForm" class="vstack gap-3">
      <div>
        <label class="form-label" for="verifyContact">Email or Phone</label>
        <input
          class="form-control form-control-lg"
          id="verifyContact"
          name="contact"
          type="text"
          inputmode="text"
          autocomplete="username"
          required
        />
      </div>

      <div>
        <label class="form-label" for="otp">OTP Code</label>
        <input
          class="form-control form-control-lg otp-input"
          id="otp"
          name="otp"
          type="text"
          maxlength="6"
          placeholder="000000"
          required
        />
      </div>

      <button class="btn btn-success btn-lg w-100" type="submit">
        <span class="spinner-icon">âœ“</span> Verify OTP
      </button>
    </form>
  </section>
```

```
        </div>

        <div class="col-12 col-xl-6">
          <section class="card-soft p-4 h-100">
            <div class="d-flex justify-content-between align-items-center mb-3">
              <h2 class="h4 mb-0">
                <span class="step-badge">3</span> User CRUD (Protected)
              </h2>
              <button id="fetchUsersBtn" class="btn btn-sm btn-outline-brand"
type="button">ðŸ"‹ Fetch Users</button>
            </div>
            <p class="small text-secondary mb-3">
              â€" Verify OTP first, then create user with the same verified
email/phone.
            </p>

            <form id="createUserForm" class="row g-3">
              <div class="col-12 col-md-6">
                <label class="form-label" for="name">Full Name</label>
                <input class="form-control" id="name" name="name" type="text"
placeholder="John Doe" required />
              </div>
              <div class="col-12 col-md-6">
                <label class="form-label" for="userContact">Email or Phone</label>
                <input
                  class="form-control"
                  id="userContact"
                  name="contact"
                  type="text"
                  placeholder="verified contact"
                  required
                />
              </div>
              <div class="col-12">
                <label class="form-label" for="contactType">Contact Type</label>
                <select class="form-select" id="contactType" name="contactType"
required>
                  <option value="">-- Select --</option>
                  <option value="email">Email</option>
                  <option value="phone">Phone</option>
                </select>
              </div>
              <div class="col-12">
                <button class="btn btn-brand w-100" type="submit">âž• Create
User</button>
              </div>
            </form>
          </section>
        </div>
```

```
        <div class="col-12 col-xl-6">
          <section class="card-soft p-4 h-100">
            <div class="d-flex justify-content-between align-items-center mb-3">
              <h2 class="h4 mb-0">Live Response</h2>
              <button id="clearLogsBtn" class="btn btn-sm btn-outline-secondary"
type="button">ðŸ—‘ Clear</button>
            </div>
            <pre id="output" class="output-box mb-0">Ready.</pre>
          </section>
        </div>

        <div class="col-12">
          <section class="card-soft p-4">
            <div class="d-flex justify-content-between align-items-center mb-3">
              <h2 class="h4 mb-0">ðŸ“œ Verification History</h2>
              <span class="badge bg-brand" id="logCount">0</span>
            </div>
            <div id="logsContainer" class="logs-list">
              <p class="text-secondary text-center py-4">No verifications yet. Start
by generating an OTP.</p>
            </div>
          </section>
        </div>
      </div>
    </main>

    <script

src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
      integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
      crossorigin="anonymous"
    ></script>
    <script src="script.js"></script>
  </body>
</html>
~~~

### File Name: frontend/style.css
Purpose: Controls complete frontend visual design, responsiveness, and animations.
~~~css
:root {
  --bg: #f8f6f3;
  --text: #1a1410;
  --muted: #6b5f53;
  --brand: #0f8b7d;
  --brand-deep: #086e61;
  --brand-light: #20c997;
```

```
  --brand-accent: #14d8a6;
  --success: #10b981;
  --warning: #f59e0b;
  --danger: #ef4444;
  --card: rgba(255, 255, 255, 0.75);
  --card-border: rgba(100, 88, 75, 0.12);
  --output-bg: #0f0f0f;
  --output-text: #fef3c7;
  --shadow-sm: 0 2px 8px rgba(15, 10, 5, 0.08);
  --shadow-md: 0 8px 24px rgba(15, 10, 5, 0.12);
  --shadow-lg: 0 20px 48px rgba(15, 10, 5, 0.16);
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-height: 100vh;
  font-family: "Manrope", sans-serif;
  color: var(--text);
  background:
    radial-gradient(circle at 15% 20%, rgba(15, 139, 125, 0.25), transparent 35%),
    radial-gradient(circle at 85% 15%, rgba(32, 201, 151, 0.15), transparent 45%),
    radial-gradient(circle at 50% 100%, rgba(15, 139, 125, 0.1), transparent 50%),
    linear-gradient(135deg, #f9f5f0 0%, #f0f9f7 50%, #eef5ff 100%);
  overflow-x: hidden;
}

.bg-orb {
  position: fixed;
  border-radius: 999px;
  filter: blur(30px);
  z-index: -1;
  opacity: 0.65;
  animation: drift 11s ease-in-out infinite alternate;
}

.bg-orb-1 {
  width: 320px;
  height: 320px;
  top: -80px;
  right: -70px;
  background: rgba(13, 148, 136, 0.35);
}

.bg-orb-2 {
  width: 300px;
```

```
  height: 300px;
  left: -70px;
  bottom: -80px;
  background: rgba(251, 191, 36, 0.34);
  animation-duration: 14s;
}

.card-soft {
  background: var(--card);
  border: 1px solid var(--card-border);
  border-radius: 1.25rem;
  box-shadow: var(--shadow-md);
  -webkit-backdrop-filter: blur(8px);
  backdrop-filter: blur(8px);
  animation: riseIn 0.7s cubic-bezier(0.34, 1.56, 0.64, 1) both;
  transition: all 0.3s cubic-bezier(0.34, 1.56, 0.64, 1);
}

.card-soft:hover {
  box-shadow: var(--shadow-lg);
  transform: translateY(-2px);
}

.hero {
  border-left: 8px solid var(--brand);
}

.eyebrow {
  color: var(--brand-deep);
  text-transform: uppercase;
  letter-spacing: 0.08em;
  font-weight: 700;
  font-size: 0.8rem;
}

.subtitle {
  max-width: 58ch;
  color: var(--muted);
}

.token-pill {
  background: #fff;
  color: var(--brand-deep);
  border: 1px solid rgba(15, 118, 110, 0.35);
  padding: 0.55rem 0.95rem;
  border-radius: 999px;
  font-weight: 700;
  font-size: 0.92rem;
  white-space: nowrap;
```

```
}

.token-pill.is-authenticated {
  background: rgba(13, 148, 136, 0.12);
}

.form-control,
.form-select {
  border-radius: 0.9rem;
  border: 2px solid rgba(120, 113, 108, 0.15);
  padding: 0.85rem 1rem;
  font-size: 1rem;
  font-weight: 500;
  transition: all 0.3s ease;
  background: rgba(255, 255, 255, 0.6);
}

.form-control:hover,
.form-select:hover {
  border-color: rgba(15, 139, 125, 0.3);
  background: rgba(255, 255, 255, 0.8);
}

.form-control:focus,
.form-select:focus {
  border-color: var(--brand);
  background: rgba(255, 255, 255, 0.95);
  box-shadow: 0 0 0 0.3rem rgba(15, 139, 125, 0.12), inset 0 0 0 2px rgba(15, 139, 125,
0.05);
  outline: none;
}

.btn-brand {
  background: linear-gradient(135deg, var(--brand) 0%, var(--brand-light) 50%, var(--
brand-accent) 100%);
  color: #fff;
  border: none;
  font-weight: 700;
  font-size: 1rem;
  letter-spacing: 0.02em;
  box-shadow: var(--shadow-md);
  transition: all 0.3s cubic-bezier(0.34, 1.56, 0.64, 1);
  position: relative;
  overflow: hidden;
}

.btn-brand::before {
  content: '';
  position: absolute;
```

```
  top: 50%;
  left: 50%;
  width: 0;
  height: 0;
  border-radius: 50%;
  background: rgba(255, 255, 255, 0.3);
  transform: translate(-50%, -50%);
  transition: width 0.6s, height 0.6s;
}

.btn-brand:hover::before {
  width: 300px;
  height: 300px;
}

.btn-brand:hover,
.btn-brand:focus {
  color: #fff;
  background: linear-gradient(135deg, var(--brand-deep) 0%, var(--brand) 50%, var(--
brand-light) 100%);
  box-shadow: 0 12px 32px rgba(15, 139, 125, 0.35);
  transform: translateY(-3px);
}

.btn-outline-brand {
  border-color: var(--brand);
  color: var(--brand);
}

.btn-outline-brand:hover {
  background: var(--brand);
  border-color: var(--brand);
  color: #fff;
}

.otp-input {
  letter-spacing: 0.2em;
  font-weight: 800;
}

.output-box {
  min-height: 350px;
  background: linear-gradient(170deg, var(--output-bg), #292524);
  color: var(--output-text);
  border-radius: 0.95rem;
  border: 1px solid rgba(245, 158, 11, 0.32);
  padding: 1rem;
  overflow-x: auto;
  white-space: pre-wrap;
```

```
  word-break: break-word;
}

@media (max-width: 768px) {
  .hero {
    border-left-width: 5px;
  }

  .output-box {
    min-height: 260px;
  }
}

@keyframes riseIn {
  from {
    opacity: 0;
    transform: translateY(20px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

@keyframes drift {
  from {
    transform: translateY(0) translateX(0) scale(1);
  }
  to {
    transform: translateY(-25px) translateX(18px) scale(1.1);
  }
}

@keyframes popIn {
  0% {
    opacity: 0;
    transform: scale(0.8);
  }
  50% {
    opacity: 1;
    transform: scale(1.05);
  }
  100% {
    opacity: 1;
    transform: scale(1);
  }
}

@keyframes shimmer {
```

```
  0% {
    background-position: -1000px 0;
  }
  100% {
    background-position: 1000px 0;
  }
}

@keyframes pulse-glow {
  0%, 100% {
    box-shadow: 0 0 20px rgba(15, 139, 125, 0);
  }
  50% {
    box-shadow: 0 0 30px rgba(15, 139, 125, 0.3);
  }
}

/* Logs section styles */
.logs-list {
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
  max-height: 400px;
  overflow-y: auto;
  padding-right: 0.5rem;
}

.logs-list::-webkit-scrollbar {
  width: 6px;
}

.logs-list::-webkit-scrollbar-track {
  background: rgba(100, 90, 77, 0.06);
  border-radius: 10px;
}

.logs-list::-webkit-scrollbar-thumb {
  background: rgba(15, 118, 110, 0.3);
  border-radius: 10px;
}

.logs-list::-webkit-scrollbar-thumb:hover {
  background: rgba(15, 118, 110, 0.5);
}

.log-item {
  background: rgba(255, 255, 255, 0.6);
  border-left: 5px solid transparent;
  border-radius: 0.85rem;
```

```
  padding: 1.1rem;
  margin-bottom: 0;
  animation: slideUp 0.45s cubic-bezier(0.34, 1.56, 0.64, 1) forwards;
  box-shadow: var(--shadow-sm);
  transition: all 0.3s cubic-bezier(0.34, 1.56, 0.64, 1);
  position: relative;
  overflow: hidden;
}

.log-item::before {
  content: '';
  position: absolute;
  top: 0;
  left: 0;
  right: 0;
  height: 1px;
  background: linear-gradient(90deg, transparent, currentColor, transparent);
  opacity: 0.1;
}

.log-item:hover {
  box-shadow: var(--shadow-md);
  transform: translateX(6px);
  border-left-color: currentColor;
}

.log-item.log-success {
  border-left-color: var(--success);
  background: linear-gradient(135deg, rgba(16, 185, 129, 0.12), rgba(16, 185, 129,
0.05));
}

.log-item.log-error {
  border-left-color: var(--danger);
  background: linear-gradient(135deg, rgba(239, 68, 68, 0.12), rgba(239, 68, 68,
0.05));
}

.log-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 0.5rem;
  gap: 1rem;
}

.log-type {
  font-weight: 700;
  font-size: 0.95rem;
```

```
  color: var(--brand-deep);
  text-transform: uppercase;
  letter-spacing: 0.05em;
}

.log-time {
  font-size: 0.85rem;
  color: var(--muted);
  white-space: nowrap;
  background: rgba(15, 118, 110, 0.08);
  padding: 0.25rem 0.6rem;
  border-radius: 0.4rem;
}

.log-details {
  display: flex;
  flex-direction: column;
  gap: 0.35rem;
}

.log-details p {
  margin: 0;
  font-size: 0.9rem;
  line-height: 1.4;
}

.log-contact {
  color: var(--text);
}

.log-contact strong {
  color: var(--brand-deep);
  font-weight: 700;
}

.log-message {
  color: var(--muted);
  font-size: 0.88rem;
}

.log-item.log-success .log-message {
  color: #059669;
}

.log-item.log-error .log-message {
  color: #dc2626;
}

@keyframes slideUp {
```

```
  from {
    opacity: 0;
    transform: translateY(12px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

@media (max-width: 768px) {
  .logs-list {
    max-height: 300px;
  }

  .log-item {
    padding: 0.8rem;
  }

  .log-header {
    flex-direction: column;
    align-items: flex-start;
    margin-bottom: 0.4rem;
  }

  .log-time {
    align-self: flex-end;
    margin-top: -0.3rem;
  }
}
/* Enhanced section headers */
section h2 {
  font-weight: 800;
  letter-spacing: -0.5px;
}

.step-badge {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: 36px;
  height: 36px;
  border-radius: 50%;
  background: linear-gradient(135deg, var(--brand) 0%, var(--brand-light) 100%);
  color: #fff;
  font-weight: 800;
  font-size: 1.1rem;
  margin-right: 0.6rem;
  box-shadow: var(--shadow-md);
```

```
  transition: all 0.3s cubic-bezier(0.34, 1.56, 0.64, 1);
  display: inline-flex;
}

.step-badge:hover {
  transform: scale(1.1) rotate(5deg);
  box-shadow: var(--shadow-lg);
}

/* Better form labels */
.form-label {
  font-weight: 700;
  color: var(--text);
  font-size: 0.95rem;
  margin-bottom: 0.6rem;
  letter-spacing: 0.3px;
  position: relative;
  padding-left: 0;
}

.form-label::after {
  content: '';
  position: absolute;
  bottom: -4px;
  left: 0;
  width: 24px;
  height: 2px;
  background: linear-gradient(90deg, var(--brand), transparent);
  border-radius: 1px;
}

/* Enhanced output box */
.output-box {
  min-height: 350px;
  background: linear-gradient(135deg, var(--output-bg) 0%, #1a1515 100%);
  color: var(--output-text);
  border-radius: 1.1rem;
  border: 1px solid rgba(245, 158, 11, 0.25);
  padding: 1.25rem;
  overflow-x: auto;
  white-space: pre-wrap;
  word-break: break-word;
  font-family: 'Monaco', 'Menlo', 'Ubuntu Mono', monospace;
  font-size: 0.9rem;
  line-height: 1.5;
  box-shadow: inset 0 2px 8px rgba(0, 0, 0, 0.3);
  transition: all 0.3s ease;
}
```

```
.output-box:hover {
  border-color: rgba(245, 158, 11, 0.4);
}

/* Enhanced token pill */
.token-pill {
  background: linear-gradient(135deg, #fff 0%, rgba(255, 255, 255, 0.9) 100%);
  color: var(--brand-deep);
  border: 1.5px solid rgba(15, 139, 125, 0.25);
  padding: 0.7rem 1.2rem;
  border-radius: 999px;
  font-weight: 800;
  font-size: 0.95rem;
  white-space: nowrap;
  box-shadow: var(--shadow-sm);
  transition: all 0.3s ease;
  animation: popIn 0.6s cubic-bezier(0.34, 1.56, 0.64, 1) both;
}

.token-pill.is-authenticated {
  background: linear-gradient(135deg, rgba(16, 185, 129, 0.15), rgba(32, 201, 151,
0.1));
  border-color: var(--success);
  color: var(--success);
  box-shadow: 0 0 20px rgba(16, 185, 129, 0.2);
}

.token-pill.is-authenticated::after {
  content: ' â€“';
  font-weight: 800;
}

/* Enhanced secondary buttons */
.btn-sm {
  font-weight: 700;
  transition: all 0.3s cubic-bezier(0.34, 1.56, 0.64, 1);
}

.btn-outline-secondary {
  border-color: rgba(107, 95, 83, 0.3);
  color: var(--muted);
}

.btn-outline-secondary:hover {
  background: rgba(107, 95, 83, 0.1);
  border-color: var(--muted);
  color: var(--text);
  transform: translateY(-2px);
  box-shadow: var(--shadow-sm);
```

```
}
/* OTP Input Enhancement */
.otp-input {
  letter-spacing: 0.3em;
  font-weight: 800;
  font-size: 1.5rem;
  text-align: center;
}

/* Responsive Media Queries */
@media (max-width: 768px) {
  .hero {
    border-left-width: 5px;
  }

  .output-box {
    min-height: 260px;
  }

  .step-badge {
    width: 32px;
    height: 32px;
    font-size: 0.95rem;
  }

  .card-soft {
    border-radius: 1rem;
  }

  .token-pill {
    padding: 0.6rem 1rem;
    font-size: 0.9rem;
  }

  .btn-brand {
    font-size: 0.95rem;
  }

  .logs-list {
    max-height: 300px;
  }
}

@media (max-width: 480px) {
  h1 {
    font-size: 1.75rem;
  }

  .form-control,
```

```
  .form-select,
  .btn {
    font-size: 1rem;
  }

  .output-box {
    padding: 0.75rem;
    min-height: 200px;
  }

  .log-item {
    padding: 0.85rem;
  }
}
.hero {
  border-left: 8px solid var(--brand);
  position: relative;
  overflow: hidden;
}

.hero::after {
  content: '';
  position: absolute;
  top: 0;
  right: 0;
  width: 200px;
  height: 200px;
  background: radial-gradient(circle, rgba(32, 201, 151, 0.1), transparent);
  border-radius: 50%;
  pointer-events: none;
}

.eyebrow {
  color: var(--brand-deep);
  text-transform: uppercase;
  letter-spacing: 0.1em;
  font-weight: 800;
  font-size: 0.8rem;
  position: relative;
  display: inline-block;
  padding-bottom: 0.5rem;
}

.eyebrow::after {
  content: '';
  position: absolute;
  bottom: 0;
  left: 0;
  width: 20px;
```

```
  height: 2px;
  background: linear-gradient(90deg, var(--brand), var(--brand-light));
  border-radius: 1px;
}

.subtitle {
  max-width: 58ch;
  color: var(--muted);
  line-height: 1.6;
  font-size: 1.05rem;
  font-weight: 500;
}
/* Container & spacing improvements */
main {
  animation: fadeIn 0.8s ease 0.2s both;
}

.row {
  animation: staggerChildren 0.8s ease-out both;
}

.row > [class*="col-"] {
  animation: riseIn 0.6s cubic-bezier(0.34, 1.56, 0.64, 1) both;
}

.row > [class*="col-"]:nth-child(1) { animation-delay: 0s; }
.row > [class*="col-"]:nth-child(2) { animation-delay: 0.1s; }
.row > [class*="col-"]:nth-child(3) { animation-delay: 0.2s; }
.row > [class*="col-"]:nth-child(4) { animation-delay: 0.3s; }
.row > [class*="col-"]:nth-child(5) { animation-delay: 0.4s; }

/* Form group improvements */
.form-group,
.mb-3 {
  animation: fadeIn 0.5s ease both;
}

/* Badge enhancements */
.badge {
  font-weight: 800;
  padding: 0.5rem 0.85rem;
  letter-spacing: 0.3px;
  border-radius: 0.6rem;
  transition: all 0.3s cubic-bezier(0.34, 1.56, 0.64, 1);
}

.badge:hover {
  transform: scale(1.05);
}
```

```
.badge.bg-brand {
  background: linear-gradient(135deg, var(--brand) 0%, var(--brand-light)
100%) !important;
  box-shadow: var(--shadow-sm);
}

/* Button group spacing */
.btn + .btn {
  margin-left: 0.5rem;
}

/* Enhanced focus visible */
:focus-visible {
  outline: 2px solid var(--brand);
  outline-offset: 2px;
  border-radius: 0.5rem;
}

/* Smooth text selection */
::selection {
  background: rgba(15, 139, 125, 0.3);
  color: var(--text);
}

::-moz-selection {
  background: rgba(15, 139, 125, 0.3);
  color: var(--text);
}

/* Better scrollbar styling */
::-webkit-scrollbar {
  width: 8px;
}

::-webkit-scrollbar-track {
  background: rgba(107, 95, 83, 0.05);
  border-radius: 10px;
}

::-webkit-scrollbar-thumb {
  background: rgba(15, 139, 125, 0.25);
  border-radius: 10px;
  transition: all 0.3s ease;
}

::-webkit-scrollbar-thumb:hover {
  background: rgba(15, 139, 125, 0.5);
}
```

```
/* Logs section */
.logs-list {
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
  max-height: 400px;
  overflow-y: auto;
  padding-right: 0.5rem;
}
@keyframes pulse-glow {
  0%, 100% {
    box-shadow: 0 0 20px rgba(15, 139, 125, 0);
  }
  50% {
    box-shadow: 0 0 30px rgba(15, 139, 125, 0.3);
  }
}

@keyframes fadeIn {
  from {
    opacity: 0;
  }
  to {
    opacity: 1;
  }
}

@keyframes staggerChildren {
  0% {
    opacity: 0;
  }
  100% {
    opacity: 1;
  }
}

@keyframes slideUp {
  from {
    opacity: 0;
    transform: translateY(16px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}
.btn-outline-secondary:hover {
  background: rgba(107, 95, 83, 0.1);
```

```
  border-color: var(--muted);
  color: var(--text);
  transform: translateY(-2px);
  box-shadow: var(--shadow-sm);
}

/* Bootstrap success button override */
.btn-success {
  background: linear-gradient(135deg, var(--success) 0%, #059669 100%);
  border: none;
  font-weight: 700;
  font-size: 1rem;
  letter-spacing: 0.02em;
  box-shadow: var(--shadow-md);
  transition: all 0.3s cubic-bezier(0.34, 1.56, 0.64, 1);
  position: relative;
  overflow: hidden;
}

.btn-success::before {
  content: '';
  position: absolute;
  top: 50%;
  left: 50%;
  width: 0;
  height: 0;
  border-radius: 50%;
  background: rgba(255, 255, 255, 0.3);
  transform: translate(-50%, -50%);
  transition: width 0.6s, height 0.6s;
}

.btn-success:hover::before {
  width: 300px;
  height: 300px;
}

.btn-success:hover,
.btn-success:focus {
  background: linear-gradient(135deg, #059669 0%, var(--success) 100%);
  box-shadow: 0 12px 32px rgba(16, 185, 129, 0.35);
  transform: translateY(-3px);
  color: #fff;
}

/* Button group enhancements */
.btn-group > .btn,
.btn-group-vertical > .btn {
  transition: all 0.3s cubic-bezier(0.34, 1.56, 0.64, 1);
```

```
}

/* Better button sizing */
.btn-lg {
  padding: 0.75rem 1.5rem;
  font-size: 1.05rem;
}

.btn-sm {
  padding: 0.4rem 0.8rem;
  font-size: 0.9rem;
}
~~~


### File Name: frontend/script.js
Purpose: Handles API calls, logs rendering, auth token state, and form interactions.
~~~js
const API_BASE = 'http://localhost:5000/api';
let authToken = '';
let verificationLogs = [];

const output = document.getElementById('output');
const tokenStatus = document.getElementById('tokenStatus');
const logsContainer = document.getElementById('logsContainer');
const logCount = document.getElementById('logCount');

const setTokenStatus = (isAuthenticated) => {
  if (!tokenStatus) return;

  if (isAuthenticated) {
    tokenStatus.textContent = 'Token: Verified';
    tokenStatus.classList.add('is-authenticated');
    return;
  }

  tokenStatus.textContent = 'Token: Not Verified';
  tokenStatus.classList.remove('is-authenticated');
};

const showOutput = (data) => {
  output.textContent = typeof data === 'string' ? data : JSON.stringify(data, null, 2);
};

const addLog = (type, contact, details, status = 'success') => {
  const timestamp = new Date().toLocaleTimeString('en-US', {
    hour: '2-digit',
    minute: '2-digit',
    second: '2-digit',
    hour12: true,
```

```
  });

  verificationLogs.unshift({
    id: Date.now(),
    type,
    contact,
    details,
    status,
    timestamp,
  });

  renderLogs();
};

const renderLogs = () => {
  if (verificationLogs.length === 0) {
    logsContainer.innerHTML = '<p class="text-secondary text-center py-4">No
verifications yet. Start by generating an OTP.</p>';
    logCount.textContent = '0';
    return;
  }

  logCount.textContent = verificationLogs.length;

  logsContainer.innerHTML = verificationLogs
    .map((log) => {
      const typeEmoji = {
        'otp-generated': 'ðŸ"¤',
        'otp-verified': 'âœ…',
        'user-created': 'ðŸ'¤',
        error: 'â Œ',
      }[log.type] || 'ðŸ" ';

      const statusClass = log.status === 'success' ? 'log-success' : 'log-error';

      return `
        <div class="log-item ${statusClass}">
          <div class="log-header">
            <span class="log-type">${typeEmoji} ${log.type.replace('-', '
').toUpperCase()}</span>
            <span class="log-time">${log.timestamp}</span>
          </div>
          <div class="log-details">
            <p class="log-contact"><strong>Contact:</strong> ${log.contact}</p>
            <p class="log-message">${log.details}</p>
          </div>
        </div>
      `;
    })
```

```
    .join('');
};

const request = async (url, options = {}) => {
  const response = await fetch(url, options);
  const data = await response.json();

  if (!response.ok) {
    throw new Error(data.message || 'Request failed');
  }

  return data;
};

document.getElementById('generateForm').addEventListener('submit', async (event) => {
  event.preventDefault();
  const generateForm = document.getElementById('generateForm');
  const contact = document.getElementById('contact').value;

  try {
    const data = await request(`${API_BASE}/auth/generate-otp`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ contact }),
    });

    showOutput(data);
    addLog('otp-generated', contact, `OTP sent successfully. Expires in 5 minutes.`);
    generateForm.reset();
    document.getElementById('verifyContact').value = data?.data?.contact || contact;
  } catch (error) {
    showOutput(error.message);
    addLog('otp-generated', contact, `Failed: ${error.message}`, 'error');
  }
});

document.getElementById('verifyForm').addEventListener('submit', async (event) => {
  event.preventDefault();

  const verifyForm = document.getElementById('verifyForm');

  const contact = document.getElementById('verifyContact').value;
  const otp = document.getElementById('otp').value;

  try {
    const data = await request(`${API_BASE}/auth/verify-otp`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ contact, otp }),
```

```
    });

    authToken = data.token;
    setTokenStatus(true);
    showOutput({ ...data, info: 'Token stored in memory for protected APIs.' });
    addLog('otp-verified', contact, `Successfully verified. Token received.`);
    verifyForm.reset();
  } catch (error) {
    showOutput(error.message);
    addLog('otp-verified', contact, `Verification failed: ${error.message}`, 'error');
  }
});

document.getElementById('fetchUsersBtn').addEventListener('click', async () => {
  try {
    const data = await request(`${API_BASE}/users`, {
      method: 'GET',
      headers: {
        Authorization: `Bearer ${authToken}`,
      },
    });

    showOutput(data);
  } catch (error) {
    showOutput(error.message);
  }
});

document.getElementById('createUserForm').addEventListener('submit', async (event) =>
{
  event.preventDefault();

  const payload = {
    name: document.getElementById('name').value,
    contact: document.getElementById('userContact').value,
    contactType: document.getElementById('contactType').value,
  };

  try {
    const data = await request(`${API_BASE}/users`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        Authorization: `Bearer ${authToken}`,
      },
      body: JSON.stringify(payload),
    });

    showOutput(data);
```

```
    addLog('user-created', payload.contact, `User "${payload.name}" created
successfully.`);
    document.getElementById('createUserForm').reset();
  } catch (error) {
    showOutput(error.message);
    addLog('user-created', payload.contact, `Failed: ${error.message}`, 'error');
  }
});

document.getElementById('clearLogsBtn').addEventListener('click', () => {
  verificationLogs = [];
  renderLogs();
});
~~~

## 4. Complete README Content

This section includes the complete README.md content for a consolidated project
document.

~~~markdown
# OTP Verification System

A complete OTP Verification System with:

- Frontend: HTML, CSS, JavaScript
- Backend: Node.js + Express
- Database: MongoDB with Mongoose
- OTP delivery: Nodemailer (email) + Twilio (SMS)
- Middleware: Validation + Authentication
- CRUD APIs: User management
- OTP APIs: Generate, Verify, Expiry

## Project Structure

```text
OTP VERIFICATION SYSTEM/
  backend/
    src/
      config/
      controllers/
      middlewares/
      models/
      routes/
      services/
      utils/
      validators/
      app.js
      server.js
```
```

```
    .env.example
    package.json
  frontend/
    index.html
    style.css
    script.js
  postman/
    OTP-System.postman_collection.json
  docs/
    screenshots/
  README.md
```

Features Implemented

1. Generate OTP (6-digit)
2. Verify OTP
3. OTP expiry handling
4. User CRUD operations (Create, Read, Update, Delete)
5. Protected routes with JWT authentication middleware
6. Input validation middleware using express-validator
7. Error handling middleware for clean API responses
8. Frontend integration with backend APIs
9. **Verification History Logs** â€" Real-time tracking of all OTP/CRUD operations with timestamps, status indicators, and auto-clearing

âœ¨ Verification Logs Feature

The frontend now includes a **ðŸ"œ Verification History** section that:
- **Tracks all operations** in real-time (OTP generation, verification, user creation)
- **Displays with timestamps** (HH:MM:SS format)
- **Color-coded status** â€" Green for success, red for errors
- **Shows details** â€" Contact, operation type, result message
- **Auto-updates count** â€" Badge shows total log entries
- **Clear button** â€" Reset all logs with one click
- **Scrollable list** â€" Handles unlimited log entries

See LOGS_FEATURE_GUIDE.md for detailed testing instructions.

API Endpoints

Public

- `GET /api/health`
- `POST /api/auth/generate-otp`
- `POST /api/auth/verify-otp`

Protected (Bearer token required)

- `GET /api/users`
- `GET /api/users/:id`
- `POST /api/users`
- `PUT /api/users/:id`
- `DELETE /api/users/:id`

Setup Instructions

1) Backend Setup

```bash
cd backend
npm install
```

Create `.env` in `backend`:

```env
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/otp_verification_system
JWT_SECRET=change_this_secret
OTP_EXPIRY_MINUTES=5
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your_email@gmail.com
SMTP_PASS=your_email_app_password
SMTP_FROM=your_email@gmail.com
TWILIO_ACCOUNT_SID=your_twilio_account_sid
TWILIO_AUTH_TOKEN=your_twilio_auth_token
TWILIO_PHONE_NUMBER=+12345678901
SMS_DEFAULT_COUNTRY_CODE=+91
```

If MongoDB is not running, app automatically falls back to in-memory store.

Run backend:

```bash
npm run dev
```

2) Frontend Setup

Open `frontend/index.html` in browser.

For best results, use Live Server extension or any static server.

Project Flow (as required)

User enters email/phone
-> Server generates 6-digit OTP
-> OTP stored temporarily with expiry timestamp
-> OTP is sent through Nodemailer (email) or Twilio (SMS)
-> User enters OTP
-> Server verifies OTP (valid, expired, or invalid)
-> Access granted (JWT token) or denied
-> JWT token is used for protected CRUD APIs.

Postman Testing

How to Run Requests in Postman

1. **Download and open Postman**
   - https://www.postman.com/downloads/

2. **Import collection**
   - Click **File â† Import**
   - Select `postman/OTP-System.postman_collection.json`
   - Click **Import**

3. **Run requests in sequence**

   Step 1: Generate OTP
   ```
   POST http://localhost:5000/api/auth/generate-otp
   Body: {
     "contact": "test@example.com"
   }
   ```
   Response: Returns 6-digit OTP code

   Step 2: Verify OTP
   ```
   POST http://localhost:5000/api/auth/verify-otp
   Body: {
     "contact": "test@example.com",
     "otp": "906830"
   }
   ```
   Response: Returns JWT token (auto-saved to collection variable)

   Step 3: Create User (protected)
   ```
   POST http://localhost:5000/api/users
   Headers: Authorization: Bearer <token>
   Body: {
     "name": "John Doe",
     "contact": "test@example.com",
   ```

```
  "contactType": "email"
}
```

Step 4: List Users (protected)
```
GET http://localhost:5000/api/users
Headers: Authorization: Bearer <token>
```

Step 5: Other CRUD Operations
- `GET /api/users/:id` - View one user
- `PUT /api/users/:id` - Update user
- `DELETE /api/users/:id` - Delete user

4. **Tips**
   - Test scripts auto-populate OTP and token variables
   - Use same verified contact for creating user
   - All CRUD endpoints require valid JWT token

Screenshots

1. Frontend page loaded
2. Generate OTP success response
3. Verify OTP success response
4. CRUD list users response
5. Error case (invalid/expired OTP)
6. Backend Server Side Responses
7. Data in Mongo Compass/Shell
8. Postman otp verification.

Notes

- OTP value is returned in API response for demo/testing only.
- In production, never return OTP in API response.
~~~
~~~